

Министерство науки и высшего образования РФ
ФГАОУ ВО «Омский государственный технический университет»
Кафедра «Автоматизированные системы обработки информации и
управления»

ОТЧЕТ

по производственной (преддипломной) практике
студента Гергерта Павла Алексеевича группы ЗИВТ-211

Пояснительная записка

Шифр работы От-20968999-06 ПЗ

Специальность 09.03.01 Информатика и вычислительная техника

Руководитель от ОмГТУ доцент,
канд. Техн. Наук, доцент

А. Г. Белик

Руководитель от кафедры
ст. преподаватель

А. Г. Анатолийев

Студент

П. А. Гергерт

Омск, 2026

Задание на преддипломную практику

Период прохождения практики: 02.02.2026-14.03.2026

Студенту ОмГТУ группы ЗИВТ-211 Гергерту Павлу Алексеевичу

Место прохождения практики: Институт среднего профессионального образования и довузовской подготовки (СПО и ДП) Омского государственного университета им. Ф.М. Достоевского

Задание руководителя от ОмГТУ доцента Белик А. Г.

1. Анализ деятельности предприятия и существующей системы автоматизации с предоставлением версии первого раздела ВКРБ.
2. Постановка задачи на проектирование по теме ВКРБ с предоставлением версии второго раздела ВКРБ.
3. Техническое проектирование автоматизированной системы, с предоставлением версии третьего раздела ВКРБ.

Задание руководителя от предприятия:

1. Изучить организационную структуру и выявить проблемы в текущих бизнес-процессах мониторинга успеваемости.
2. Собрать требования подразделений к отчетам по итогам выполнения студентами заданий.
3. Согласовать предложения и описать архитектуру программного средства.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 Анализ процесса мониторинга успеваемости как объекта автоматизации ...	6
1.1 Мониторинг успеваемости как предметная область исследования.....	6
1.2 Анализ концепции систем автоматизации мониторинга успеваемости.....	7
1.3 Анализ систем автоматизации мониторинга успеваемости	10
1.3.1 Яндекс Формы и инструменты визуализации.....	10
1.3.2 Система управления обучением MOODLE	13
1.3.3 Сервис Online Test Pad.....	15
1.3.4 Сравнительный анализ систем.....	19
1.3.5 Обоснование собственной разработки.....	21
2 Постановка задачи на проектирование и разработку системы	23
2.1 Основные характеристики разработки	23
2.2 Ограничения и рекомендации к разработке системы	24
2.3 Требования к разработке системы.....	25
2.4 Структура входной и выходной информации	26
ЗАКЛЮЧЕНИЕ	42

ВВЕДЕНИЕ

В условиях активной цифровизации системы образования особую значимость приобретает внедрение информационных технологий в процессы контроля и оценки учебных достижений обучающихся. Современные образовательные организации функционируют в среде, характеризующейся увеличением объема учебной информации, необходимостью регулярного мониторинга результатов обучения и повышением требований к объективности и прозрачности оценивания. Традиционные формы контроля знаний, основанные на ручной проверке работ и фрагментарном анализе результатов, а также уведомление заинтересованных сторон о проведенном контроле и его результатах [1] не позволяют в полной мере обеспечить системный и оперативный анализ успеваемости на уровне отдельного обучающегося, учебной группы и параллели.

Развитие цифровых образовательных платформ и систем дистанционного обучения создало предпосылки для автоматизации процедур тестирования и фиксации результатов. Однако существующие решения, как правило, ориентированы либо на организацию учебного процесса в целом, либо на проведение разовых контрольных мероприятий, не обеспечивая комплексного мониторинга динамики освоения учебного материала [2]. В условиях перехода к компетентностной модели образования возрастает потребность в инструментах, позволяющих выявлять проблемные темы, анализировать типичные ошибки и формировать индивидуальные образовательные траектории.

Актуальность темы выпускной квалификационной работы обусловлена необходимостью разработки специализированной автоматизированной системы мониторинга успеваемости обучающихся, обеспечивающей не только проведение тестирования, но и накопление, хранение и аналитическую обработку результатов. Такая система должна способствовать повышению объективности оценки знаний, снижению трудозатрат преподавателя и созданию условий для адаптивного подбора учебных заданий на основе статистики выполнения.

Объектом исследования является процесс мониторинга учебных достижений обучающихся в образовательной организации. Предметом исследования выступает автоматизированная система, предназначенная для организации тестирования, хранения результатов и анализа успеваемости.

Целью работы является разработка автоматизированной системы мониторинга успеваемости обучающихся, обеспечивающей сбор, хранение и аналитическую обработку данных о результатах тестирования, а также реализацию механизма адаптивного подбора заданий. Для достижения поставленной цели предполагается провести анализ существующих решений, сформулировать требования к разрабатываемой системе, разработать архитектуру программного средства и реализовать его основные функциональные модули.

Результатом работы должна стать информационная система, ориентированная на комплексный мониторинг учебной деятельности и поддерживающая анализ успеваемости на различных уровнях.

Для достижения поставленной цели в рамках выпускной квалификационной работы предполагается решение комплекса взаимосвязанных задач: проведение анализа предметной области; изучение особенностей организации контроля знаний в образовательной организации; определение существующих проблем и ограничений различных методов оценки; выполнение обзора и сравнительного анализа существующих программных решений; формирование требований к разрабатываемой системе (функциональных и нефункциональных), разработка структуры входной и выходной информации, описание ролей пользователей и сценариев взаимодействия; разработка архитектуры программного средства, структуры базы данных, выбор технологического стека и непосредственно сама разработка; тестирование работоспособности системы и анализа полученных результатов; оценка достигнутых результатов и определение вектора дальнейшего развития системы.

Структурно выпускная квалификационная работа состоит из введения, основной части, заключения и списка использованных источников. В первой главе рассматриваются теоретические основы мониторинга успеваемости и проводится анализ существующих аналогов. Во второй главе осуществляется постановка задачи проектирования и формируются требования к системе. Третья глава посвящена разработке архитектурных решений, моделированию и описанию алгоритмов работы системы. В четвертой главе обосновывается выбор программного и технического обеспечения, а также рассматриваются особенности реализации. В заключении приводятся выводы по результатам проведенного исследования и разработки.

1 Анализ процесса мониторинга успеваемости как объекта автоматизации

1.1 Мониторинг успеваемости как предметная область исследования

Мониторинг успеваемости обучающихся представляет собой систематическую, непрерывно осуществляемую диагностическую деятельность, направленную на оценивание уровня освоения учебного материала, выявление пробелов в знаниях, умений и навыках, а также на анализ динамики учебных достижений.

Данная деятельность рассматривается не как разовая проверка, а как регулярное наблюдение за реализацией образовательных целей и показателей, сопоставление фактических результатов с заданными эталонами и принятие соответствующих педагогических решений на основе полученной информации. Такое определение мониторинга отражает его диагностическую и прогностическую функции в образовательном процессе и основано на педагогической концепции наблюдения за результатами обучения в динамике.

В педагогической науке мониторинг (контроль) успеваемости понимается как совокупность диагностических процедур, направленных на объективное отслеживание уровня обученности и качества знаний учащихся на различных этапах обучения [3]. Он включает в себя анализ текущих, тематических и итоговых результатов, сопоставление их с образовательными стандартами и учебными целями, а также оценку эффективности применяемых педагогических методов. Такое исследование позволяет получить достоверную информацию о том, насколько успешно обучающиеся усвоили образовательный материал, какие разделы вызывают затруднения и требуют дополнительного внимания со стороны преподавателя.

Мониторинг успеваемости выполняет несколько взаимосвязанных функций. Прежде всего он обеспечивает диагностику уровня знаний и умений, что позволяет преподавателю не только зафиксировать итоговую оценку, но и подробно изучить структуру ошибок и причинные факторы их возникновения. Во-вторых, мониторинг служит средством анализа эффективности педагогических стратегий, поскольку сопоставление результатов тестов и методик преподавания дает представление о том, насколько выбранные подходы способствуют достижению образовательных целей. В-третьих, он

выполняет прогностическую функцию, позволяя прогнозировать дальнейший образовательный путь обучающегося, выявляя потенциально проблемные зоны и области, требующие коррекции.

Систематический мониторинг успеваемости также играет важную роль в управлении образовательным процессом на уровне учебной организации. Сбор, хранение и обработка данных о результатах обучения создают информационную базу, необходимую для стратегического планирования, оптимизации учебных программ и принятия управленческих решений [4]. Такие данные могут использоваться при распределении ресурсов, корректировке учебного плана или формировании индивидуальных образовательных траекторий обучающихся.

В современной практике педагогические исследования подчеркивают значимость мониторинга как средства сопровождения и сопровождения достижений результатов обучающихся, то есть как инструмента не только диагностики, но и активного сопровождения учебного процесса. Мониторинг включает в себя анализ результатов на всех этапах обучения, оценку приобретенных компетенций и умения применять знания в практических задачах, а также построение карт динамики образовательных достижений в разрезе отдельных учебных тем.

Таким образом, объект автоматизации – мониторинг успеваемости – включает в себя сложный педагогический процесс, ориентированный на регулярное измерение и анализ знаний, умений и навыков обучающихся в разных разрезах: индивидуальном, групповом и программном. Автоматизированная система мониторинга должна обеспечивать поддержку всех ключевых этапов этого процесса: от сбора диагностических данных до анализа их в разрезе динамики, проблемных областей и прогнозирования образовательных результатов.

1.2 Анализ концепции систем автоматизации мониторинга успеваемости

В мировом образовательном сообществе в последние годы наблюдается устойчивый тренд к цифровизации всех уровней обучения. Цифровые технологии внедряются в учебный процесс в качестве ключевого инструмента трансформации традиционной школы и вузов, обеспечивая доступ к

образовательным ресурсам через интернет, использование электронных учебников и адаптивных систем обучения. Такие тенденции рассматриваются в научной литературе как проявление глобальной цифровой трансформации образования, обусловленной бурным развитием информационно-коммуникационных технологий, необходимость повышения цифровых компетенций у обучающихся и педагогов, а также соответствием образовательных практик современным социально-технологическим требованиям [5].

В российском контексте цифровизация образовательной системы также признана приоритетным направлением государственной политики. В России формируются цифровые инфраструктуры для поддержки электронного обучения, развивается государственный проект по созданию цифровой образовательной среды, а образовательные организации внедряют цифровые сервисы и платформы для организации учебного процесса [6]. Анализ российских исследований показывает, что цифровизация рассматривается как средство повышения доступности, эффективности и качества образования при одновременном сохранении традиционных педагогических задач.

Цифровизация образовательной среды закономерно привела к появлению специализированных информационных систем, предназначенных для автоматизации процедур контроля и анализа результатов обучения.

В научной и методической литературе автоматизированные системы мониторинга успеваемости обучающихся рассматриваются как разновидность информационных систем управления образовательным процессом, обеспечивающих сбор, хранение, обработку и анализ данных о результатах обучения. Такой подход отражен в исследованиях по педагогическому мониторингу, где подчеркивается его диагностическая, аналитическая и прогностическая функции [7]

Рассмотрим подробнее функции, реализующиеся подобными системами:

– сбор и хранение информации: автоматизированные системы мониторинга обеспечивают централизованный сбор данных о результатах обучения (оценки, посещаемость, участие в активностях, тестовые результаты) из различных источников образовательного процесса. Такие системы позволяют не только аккумулировать данные, но и структурировать

их для дальнейшего использования в аналитических модулях и отчетности, обеспечивая целостность и доступность информации.

– обработка данных: обработка данных в этих системах включает очистку и приведение данных к единому формату, агрегацию по ключевым показателям (например, средний балл, динамика успеваемости), преобразование данных в пригодные для анализа форматы (таблицы, метрики, сводки), использование алгоритмов классификации и прогнозирования для оценки успеваемости. Современные подходы включают также применение методов data mining и машинного обучения для более точной диагностики динамики обучения [8];

– анализ данных: аналитический модуль автоматизированных систем мониторинга предназначен для выявления закономерностей, отклонений и трендов в успеваемости обучающихся. Это включает: построение графиков и выводов по динамике успеваемости, выявление корреляций между различными показателями учебной деятельности [7], прогнозирование вероятности успешного завершения курса или рисков отставания, формирование рекомендаций для педагогов на основе анализа данных. Анализ данных не только позволяет оценивать качество обучения, но и служит основой для принятия решений на уровне администрации образовательной организации.

Архитектурно такие системы, как правило, строятся по клиент-серверному принципу с централизованным хранением данных [9]. Это обусловлено необходимостью обеспечения одновременного доступа пользователей, защиты персональных данных и формирования агрегированной статистики на уровне группы или образовательной организации.

Таким образом, концепция систем автоматизации мониторинга успеваемости представляет собой целостную методологию управления образовательными результатами, основанную на интеграции трех ключевых компонентов: централизованного сбора и хранения данных об учебных достижениях, их многомерной обработки с применением аналитических алгоритмов и последующей интерпретации выявленных закономерностей для диагностики образовательного процесса. Архитектурно эта концепция воплощается в клиент-серверных решениях с централизованным хранилищем

данных, что обеспечивает не только техническую реализацию сбора и анализа информации, но и соблюдение требований защиты персональных данных, а также возможность масштабирования системы от уровня отдельной учебной группы до образовательной организации в целом, превращая тем самым автоматизированный мониторинг в фундаментальный механизм повышения качества и эффективности обучения в условиях цифровой трансформации образования.

1.3 Анализ систем автоматизации мониторинга успеваемости

1.3.1 Яндекс Формы и инструменты визуализации

Яндекс Формы представляет собой облачный сервис для создания опросов, анкет и тестов, который может быть использован в образовательной практике в качестве инструмента автоматизации текущего и итогового контроля знаний. Сервис входит в экосистему цифровых инструментов Яндекса и доступен через веб-интерфейс, что позволяет применять его без установки специализированного программного обеспечения.

В контексте мониторинга успеваемости Яндекс Формы обеспечивают преподавателю возможность конструирования тестовых заданий различных типов, включая вопросы с выбором одного или нескольких вариантов ответа, открытые вопросы, вопросы в формате выпадающего списка и другие (рис. 1).

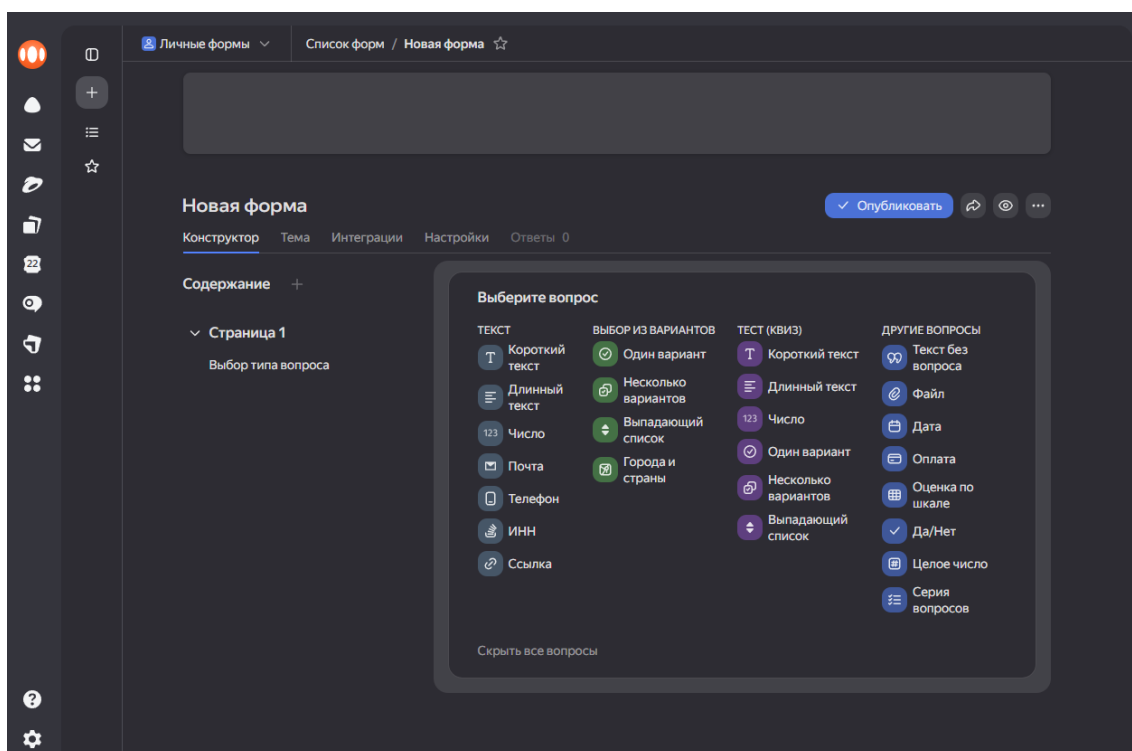
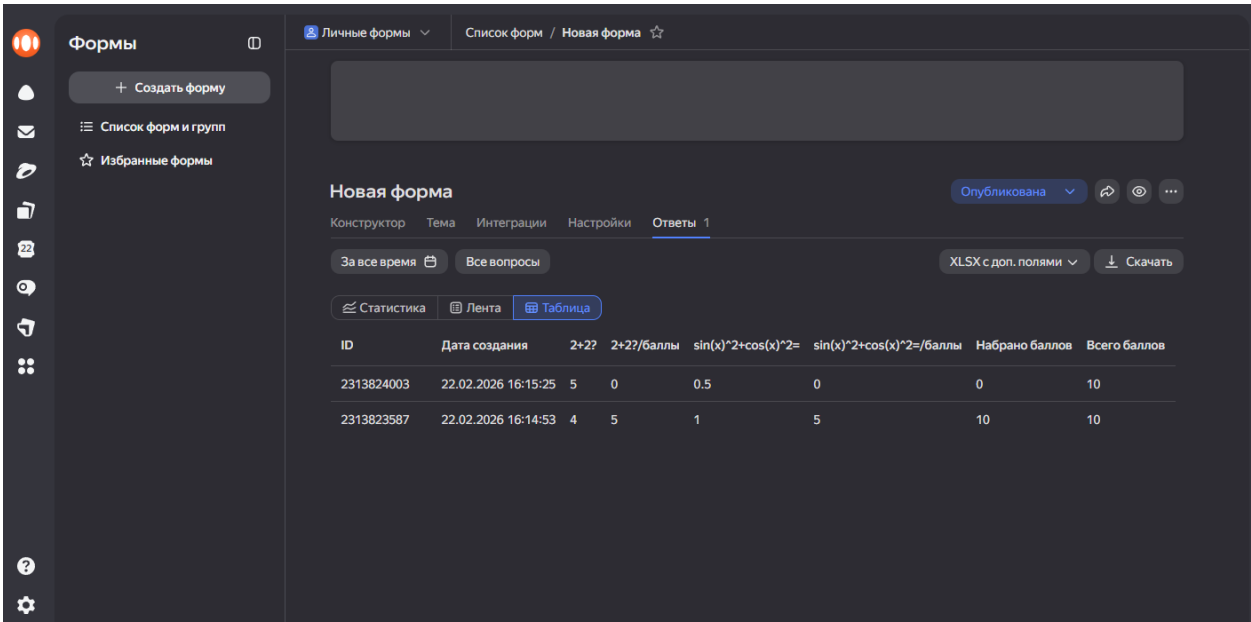


Рисунок 1 – Варианты заданий в Яндекс Формах

При создании теста преподаватель может задать правильные ответы и назначить количество баллов, что позволяет реализовать автоматический подсчет результатов. Таким образом, часть процедур проверки знаний переводится в автоматизированный режим.

После прохождения теста обучающимися система сохраняет их ответы и формирует сводную таблицу результатов. Данные могут быть представлены в табличном виде, что обеспечивает первичный статистический анализ (рис. 2). Преподаватель получает информацию о количестве набранных баллов, распределении ответов и частоте выбора каждого варианта (при выборе варианта отображения «Статистика»). Это позволяет выявить задания, вызвавшие наибольшие затруднения, а также определить общий уровень освоения темы.



ID	Дата создания	2+2?	2+2?/баллы	$\sin(x)^2 + \cos(x)^2 =$	$\sin(x)^2 + \cos(x)^2 =$ /баллы	Набрано баллов	Всего баллов
2313824003	22.02.2026 16:15:25	5	0	0.5	0	0	10
2313823587	22.02.2026 16:14:53	4	5	1	5	10	10

Рисунок 2 – Результаты прохождения тестирования в Яндекс Формы

Для расширения аналитических возможностей результаты могут быть экспортированы в электронные таблицы и обработаны с использованием встроенных средств визуализации (рис. 3).

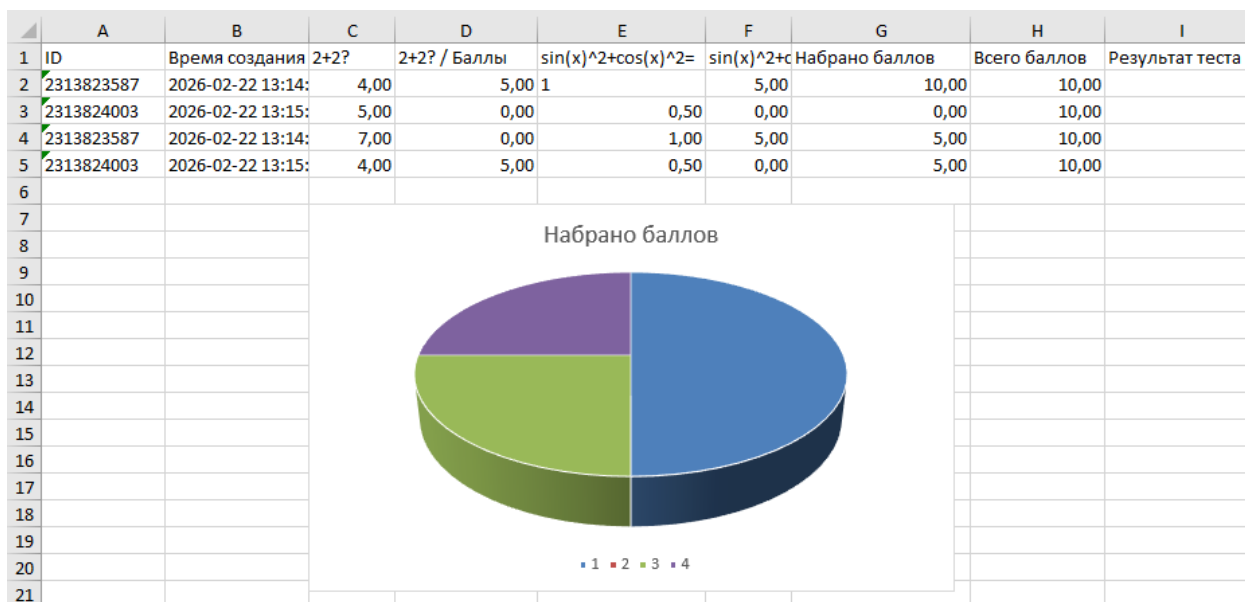


Рисунок 3 – Результаты теста, экспортированные в MS Excel и представленные в виде круговой диаграммы

На основе накопленных данных возможно построение диаграмм распределения баллов, расчет среднего значения по группе и анализ процента успешности выполнения конкретных заданий. Тем не менее подобная аналитика требует дополнительной настройки и не является полностью автоматизированной.

Архитектурно Яндекс Формы реализованы по облачному принципу. Хранение данных осуществляется на серверной стороне, а доступ к формам и результатам обеспечивается через веб-интерфейс. Такой подход позволяет организовать удаленный доступ обучающихся к тестам и обеспечивает централизованное хранение информации. Однако сервис не предусматривает сложной ролевой модели образовательной организации и не интегрирован в единую систему управления учебным процессом без использования дополнительных инструментов.

Следует отметить, что Яндекс Формы ориентированы прежде всего на проведение отдельных контрольных мероприятий, а не на комплексный мониторинг динамики успеваемости. Сервис не содержит встроенных механизмов адаптивного подбора заданий, не формирует автоматически аналитические отчеты в разрезе «обучающийся – группа – параллель» и не реализует расширенные алгоритмы анализа образовательных данных. В связи с этим его использование можно рассматривать как базовый уровень автоматизации тестирования и первичной обработки результатов, который

может служить отправной точкой для разработки специализированной системы мониторинга успеваемости обучающихся.

1.3.2 Система управления обучением MOODLE

Moodle представляет собой свободно распространяемую систему управления обучением, предназначенную для организации электронного и смешанного образовательного процесса. Платформа широко применяется в школах, колледжах и вузах и рассматривается как комплексная цифровая образовательная среда, объединяющая инструменты размещения учебных материалов, коммуникации, контроля знаний и анализа результатов.

В контексте мониторинга успеваемости Moodle обладает значительно более широкими возможностями по сравнению с облачными сервисами для создания форм. Система позволяет формировать структурированный банк заданий (рис. 4), организованный по категориям и темам, что обеспечивает повторное использование тестовых материалов и их систематизацию.

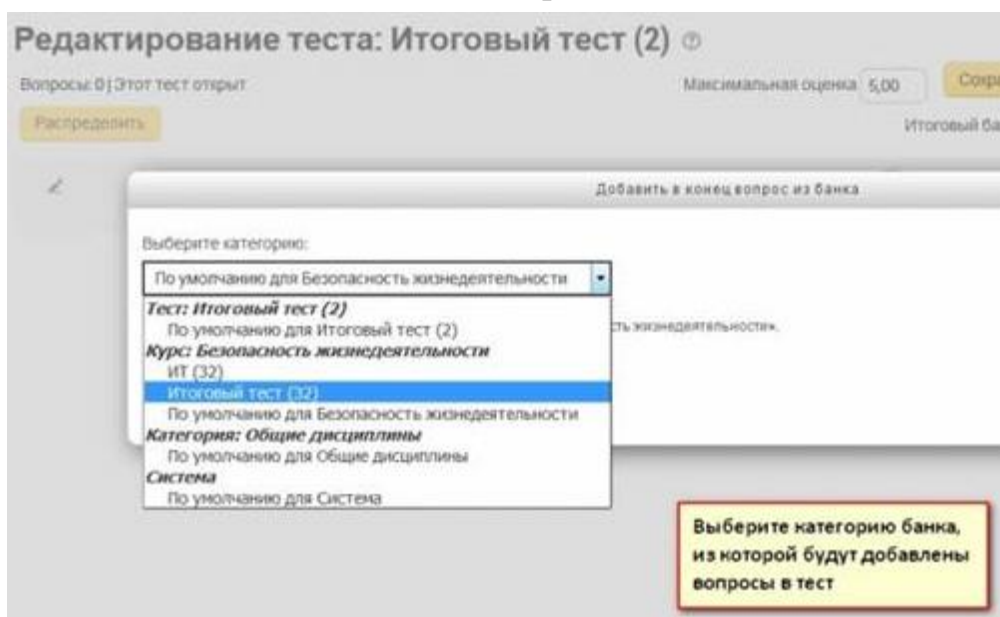


Рисунок 4 – Добавление вопроса из банка

Поддерживаются различные типы вопросов, включая множественный выбор, числовые и текстовые ответы, задания на соответствие, встроенные вычисляемые вопросы и другие форматы (рис. 5).

Управление типами вопросов

Тип вопроса	Кол-во вопросов	Версия	Необходимо	Доступно?	Настройки	Удалить
☰ Множественный выбор	3037 (+56 скрыто)	2014111000		👁️ ⬆️ ⬇️		
☐ Краткий ответ	4036 (+37 скрыто)	2014111000		👁️ ⬆️ ⬇️		
☒ Вложенные ответы (Cloze)	263 (+3 скрыто)	2014111000	Множественный выбор, Числовой ответ, Краткий ответ	👁️ ⬆️ ⬇️		
•• Верно/Неверно	131	2014111000		👁️ ⬆️ ⬇️		
▼ Выбор пропущенных слов	88	2015031800		👁️ ⬆️ ⬇️		
☒ На соответствие	69 (+2 скрыто)	2014111000		👁️ ⬆️ ⬇️		
✚ Перетащить на изображение	27	2015051000	Выбор пропущенных слов	👁️ ⬆️ ⬇️		
⌵ Garfill	24	2006082515		👁️ ⬆️ ⬇️	Настройки	
🔍 Перетаскивание в текст	16 (+2 скрыто)	2015031800	Выбор пропущенных слов	👁️ ⬆️ ⬇️		
➡ Упорядочение	18	2015071323		👁️ ⬆️ ⬇️		

Рисунок 5 – Типы вопросов в Moodle

Процесс тестирования в Moodle настраивается достаточно гибко. Преподаватель может задавать временные ограничения, количество попыток, порядок отображения вопросов, условия доступа к тесту и критерии оценивания. Результаты прохождения автоматически сохраняются в электронном журнале оценок, который интегрирован в структуру курса. Это позволяет вести учет успеваемости по всем видам деятельности обучающегося в рамках конкретной дисциплины.

Система предоставляет инструменты статистического анализа (рис. 6).

	First name / Surname	Email address	State	Started on	Completed	Time taken	Grade/10.00	Q. 1	Q. 2	Q. 3	Q. 4	Q. 5	Q. 6	Q. 7	Q. 8
☐	Sally Student	student2@example.com	In progress	10 October 2018 3:19 AM	-	-	-	0.83	-	0.00	-	-	-	0.83	-
☐	Carla Hague	CarlaHague@example.com	Finished	7 January 2021 8:36 PM	7 January 2021 8:37 PM	59 secs	6.14	0.83	0.00	0.83	0.56	0.00	0.83	0.00	0.83
☐	Fannie Asif	FannieAsif@example.com	Finished	7 January 2021 8:32 PM	7 January 2021 8:34 PM	24 secs	4.33	-	0.33	0.83	0.00	0.00	0.83	0.00	0.83
☐	Sam Student	student@example.com	Finished	9 October 2018 11:00 PM	9 October 2018 11:03 PM	2 mins 58 secs	6.11	0.83	0.00	0.00	0.28	0.00	0.00	0.83	0.83
☐	Rachael Zachary	RachaelZachary@example.com	Finished	7 January 2021 8:34 PM	7 January 2021 8:36 PM	17 secs	5.56	0.83	0.00	0.00	0.56	0.00	0.00	0.83	0.83
	Overall average						5.53 (4)	0.67 (5)	0.08 (4)	0.33 (5)	0.35 (4)	0.00 (4)	0.42 (4)	0.50 (5)	0.83 (4)

Рисунок 6 – Результаты прохождения тестирования в Moodle

Преподавателю доступны отчеты о попытках прохождения тестов, распределении баллов, средней оценке по группе и частоте выбора каждого варианта ответа. Также можно просматривать детализированную информацию по каждому обучающемуся и анализировать динамику выполнения заданий. Эти возможности позволяют выявлять проблемные темы и оценивать степень усвоения учебного материала.

Архитектурно Moodle реализована по клиент-серверной модели. Серверная часть отвечает за хранение данных, обработку запросов и реализацию прикладной логики, а пользовательский доступ осуществляется через веб-интерфейс. Такая архитектура обеспечивает централизованное хранение информации, масштабируемость и одновременную работу большого числа пользователей. Дополнительным преимуществом является модульный принцип построения, позволяющий расширять функциональность системы за счет подключаемых плагинов.

Несмотря на развитые возможности тестирования и отчетности, Moodle ориентирована прежде всего на организацию учебного процесса в целом, а не исключительно на специализированный мониторинг успеваемости. Аналитика в базовой версии преимущественно носит статистический характер и не включает сложные алгоритмы адаптивного подбора заданий или интеллектуального анализа образовательных данных. Тем не менее система обеспечивает высокий уровень автоматизации контроля знаний и может рассматриваться как полнофункциональная цифровая платформа с элементами мониторинга успеваемости.

1.3.3 Сервис Online Test Pad

Online Test Pad представляет собой облачный веб-сервис, предназначенный для создания и проведения онлайн-тестов, опросов и контрольных работ. Платформа ориентирована преимущественно на оперативную проверку знаний (стоит обратить внимание на количество существующих тестов – рис. 7) и широко используется преподавателями, репетиторами и образовательными организациями в рамках дистанционного и смешанного обучения. Сервис не требует установки программного обеспечения и доступен через браузер, что упрощает его внедрение в учебный процесс.

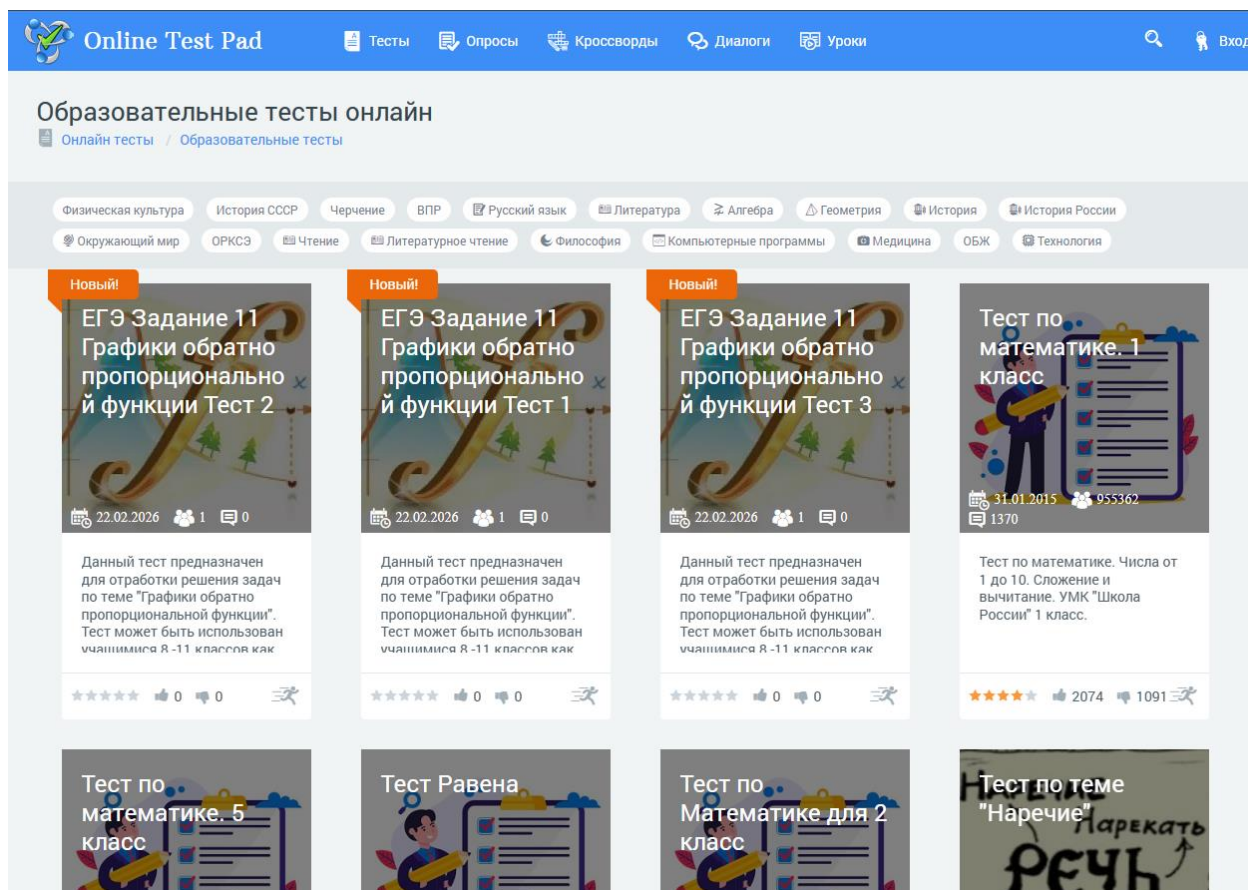


Рисунок 7 – Открытые существующие тесты на портале

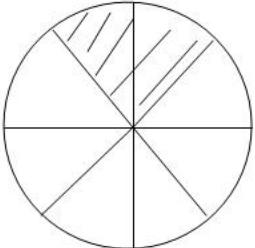
В контексте мониторинга успеваемости Online Test Pad предоставляет базовый, но удобный инструмент для оценки результатов обучения. Преподаватель может создавать тесты различной сложности, используя конструктор заданий, и формировать наборы вопросов для текущего и итогового контроля. В отличие от полнофункциональных систем управления обучением, платформа не предполагает построения развернутой структуры курса, однако позволяет быстро организовать проверку знаний по отдельным темам и модулям.

Сервис поддерживает основные типы тестовых заданий, включая вопросы с выбором одного или нескольких вариантов ответа, задания на соответствие, ввод текстового или числового ответа, а также вопросы с автоматической проверкой (рис. 8).


Online Test Pad
Тесты
Опросы
Кроссворды
Диалоги
Уроки

Тест по математике. 5 класс

44 из 18



Какая часть фигуры является закрашенной?

☐ $\frac{1}{4}$
☐ $\frac{3}{4}$
☐ $\frac{1}{8}$
☐ $\frac{2}{5}$

Далее
Завершить

Рисунок 8 – Вопрос с иллюстрацией и выбором одного ответа

Процесс тестирования настраивается достаточно гибко: можно задавать ограничения по времени, количество попыток, порядок отображения вопросов и варианты оценивания результатов. Доступ к тестам может осуществляться по прямой ссылке или с использованием кодов доступа, что удобно при работе с группами обучающихся.

Результаты прохождения тестов обрабатываются автоматически и отображаются в виде сводных таблиц (рис. 9) и диаграмм.

#	Пользователь	IP	Дата завершения	Потрачено времени	Количество правильных ответов	Процент правильных ответов (%)	Ваша оценка: молодец
27958254	Бурназян Эдуард Ахатович	62.183.124.143	19.01.2019 03:55	00:02:06	17	94.44	5
27958227	Баранцов Владимир Борисович	62.183.124.143	19.01.2019 03:54	00:02:06	18	100	5
27958224	Абрамова Марина Игоревна	62.183.124.143	19.01.2019 03:50	00:02:17	13	72.22	4
27945741	Кравченко Антон	62.183.124.180	18.01.2019 13:53	00:03:59	18	100	5
27942039	Яровикова Варвара Олеговна	62.183.124.180	18.01.2019 11:51	00:02:12	18	100	5
27941874	Клейменова Анастасия Владимировна	62.183.124.180	18.01.2019 11:45	00:03:40	12	66.67	4
27941369	Баринев	62.183.124.180	18.01.2019 11:30	00:03:11	13	72.22	4

Рисунок 9 – Результаты прохождения теста в формате таблицы

Преподавателю доступны данные о количестве попыток, набранных баллах, проценте правильных ответов и среднем результате по группе. Также возможно просматривать индивидуальные результаты каждого обучающегося, что позволяет осуществлять текущий контроль и выявлять типичные ошибки. Аналитические возможности сервиса носят преимущественно описательный характер и ориентированы на быстрый анализ итогов тестирования.

Архитектурно Online Test Pad реализован как облачное клиент-серверное решение. Хранение данных и обработка результатов выполняются на стороне сервера, а взаимодействие пользователей с системой осуществляется через веб-интерфейс. Такой подход обеспечивает доступность сервиса с различных устройств и не требует специализированной технической поддержки со стороны образовательной организации.

В целом Online Test Pad ориентирован на решение узкой задачи – оперативного контроля и самопроверки знаний. Платформа не предоставляет развитых средств управления учебным процессом и глубокой аналитики успеваемости, однако за счет простоты использования и быстроты создания

тестов может рассматриваться как удобный инструмент мониторинга результатов обучения на базовом уровне.

1.3.4 Сравнительный анализ систем

Для обоснования необходимости разработки специализированной системы автоматизации мониторинга успеваемости обучающихся целесообразно провести сравнительный анализ существующих цифровых инструментов, применяемых для контроля и оценки образовательных результатов. Сравнение функциональных, архитектурных и аналитических возможностей используемых сервисов позволяет выявить их ограничения в контексте комплексного мониторинга динамики успеваемости, а также определить требования к целевой системе. Такой подход обеспечивает обоснованный переход от анализа готовых решений к формированию концепции и функциональных характеристик разрабатываемой системы мониторинга. Сравнение ранее рассмотренных систем приведено в таблице 1.

Таблица 1 – Сравнение систем с функциями мониторинга успеваемости

Критерий	Яндекс Формы	Moodle	Online Test Pad	Целевая система
Назначение системы	Проведение отдельных опросов, анкет и тестов	Комплексная система управления обучением	Оперативная проверка знаний	Специализированная система мониторинга успеваемости
Тип архитектуры	Облачный веб-сервис	Клиент-серверная	Облачная клиент-серверная	Клиент-серверная или гибридная архитектура с централизованным хранилищем
Необходимость установки ПО	Не требуется	Требуется серверное развертывание	Не требуется	Веб-ориентированное решение без обязательной установки клиентского ПО
Типы заданий	Тестовые и открытые вопросы базового уровня	Расширенный набор типов заданий	Основные типы тестовых заданий	Поддержка расширенного набора заданий с автоматической проверкой

Продолжение таблицы 1

Критерий	Яндекс Формы	Moodle	Online Test Pad	Целевая система
Банк заданий	Отсутствует	Структуриро- ванный банк вопросов	Ограничен- ный	Наличие централизован- ного банка заданий с классификацией
Автоматическая проверка и оценивание	Частичная автоматиза- ция	Полная автоматизация	Полная автоматиза- ция	Полная автоматизация проверки и начисления баллов
Настройки процесса тестирования	Минималь- ные	Гибкие и расширенные	Средний уровень гибкости	Гибкая настройка условий тестиро- вания и оценивания
Хранение результатов	Табличные данные по отдельным формам	Электронный журнал оценок	Таблицы и диаграммы	Централизован- ное долговремен- ное хранение результатов
Аналитические возможности	Базовая статистика	Статистические отчеты	Описательная аналитика	Встроенная автоматизирован- ная аналитика без ручной обработки
Мониторинг динамики успеваемости	Не реализован	Реализован в рамках курса	Ограничен- ный	Полноценный мониторинг динамики по обучающимся и группам
Ролевая модель	Отсутствует	Развитая ролевая система	Минимальная	Многоуровневая ролевая модель пользователей
Поддержка сложной аналитики	Не поддержи- вается	Отсутствует в базовой версии	Не поддерживает ся	Поддержка расширенной образовательной аналитики
Потенциал как основа для системы мониторинга	Низкий	Высокий	Средний	Полное соответ- ствие задачам мониторинга

Проведенный сравнительный анализ облачных сервисов тестирования и систем управления обучением показал, что рассмотренные инструменты в различной степени решают задачи автоматизации контроля знаний, однако их потенциал в области комплексного мониторинга успеваемости существенно различается.

Яндекс Формы и Online Test Pad ориентированы преимущественно на проведение отдельных контрольных мероприятий и обеспечивают автоматизацию проверки и первичную обработку результатов. Вместе с тем данные сервисы не поддерживают централизованное накопление информации об учебных достижениях, не реализуют мониторинг динамики успеваемости и не предоставляют развитых аналитических инструментов. В связи с этим их использование может рассматриваться лишь как базовый уровень автоматизации контроля знаний, но не как полноценная система мониторинга.

Наиболее близкой к требованиям системы мониторинга успеваемости является Moodle. Платформа обеспечивает централизованное хранение результатов, ведение электронного журнала оценок, наличие банка заданий и встроенных средств статистического анализа. Клиент-серверная архитектура и модульный принцип построения позволяют масштабировать систему и расширять ее функциональность. В то же время аналитические возможности Moodle в базовой конфигурации носят преимущественно статистический характер и не ориентированы на глубокий анализ динамики успеваемости и интеллектуальную обработку образовательных данных.

1.3.5 Обоснование собственной разработки

В предыдущем подразделе был проведен сравнительный анализ существующих цифровых инструментов автоматизации контроля знаний и мониторинга образовательных результатов. Полученные результаты позволяют перейти к обоснованию целесообразности разработки специализированной системы мониторинга успеваемости обучающихся.

Как показал проведенный анализ, основные пользователи рассматриваемых систем сталкиваются с рядом проблем при использовании существующих решений. Эти проблемы обусловлены тем, что большинство инструментов ориентированы на проведение отдельных контрольных мероприятий, а не на комплексный анализ учебных достижений и динамики успеваемости.

К ключевым проблемам пользователей существующих систем относятся:

- отсутствие единого централизованного хранилища данных о результатах учебной деятельности обучающихся за длительный период;

- необходимость ручного сопоставления результатов различных контрольных мероприятий для оценки динамики успеваемости;
- отсутствие автоматического формирования аналитических отчетов в разрезе «обучающийся – группа – учебный период»;
- ограниченные возможности выявления проблемных тем и типичных ошибок на основе накопленных данных;
- высокая трудоемкость подготовки обобщенной аналитической информации для управленческих решений;
- зависимость глубины анализа от дополнительных инструментов и ручной обработки данных.

Дерево проблем пользователей приведено на рисунке 10.



Рисунок 10 – Дерево проблем пользователей

Несмотря на то, что системы управления обучением частично снижают влияние указанных проблем за счет централизованного хранения данных и встроенных средств отчетности, результаты сравнительного анализа показали, что их аналитические возможности носят преимущественно статистический характер и не ориентированы на задачи специализированного мониторинга успеваемости. Реализация расширенного анализа динамики учебных достижений требует дополнительной настройки системы или подключения сторонних модулей, что усложняет процесс эксплуатации.

Таким образом, выявленные в ходе сравнительного анализа проблемы пользователей подтверждают необходимость разработки специализированной системы мониторинга успеваемости обучающихся. Целевая система должна быть ориентирована на автоматизированный сбор, хранение и анализ образовательных данных, а также на формирование аналитической отчетности, обеспечивающей поддержку принятия управленческих решений. Это и обосновывает целесообразность собственной разработки.

2 Постановка задачи на проектирование и разработку системы

2.1 Основные характеристики разработки

Объектом исследования работы является процесс мониторинга успеваемости обучающихся в образовательной организации.

Предметом исследования выступает система автоматизации мониторинга успеваемости, обеспечивающая сбор, хранение и аналитическую обработку результатов учебной деятельности обучающихся.

Цель работы заключается в повышении эффективности деятельности преподавателей и административного персонала образовательной организации за счет разработки и внедрения автоматизированной системы мониторинга успеваемости, ориентированной на анализ динамики образовательных результатов и формирование аналитической отчетности.

На основе проведенного анализа существующих систем контроля знаний и управления обучением была сформулирована задача проектирования и разработки специализированной системы мониторинга успеваемости. Требуется разработать функциональный набор программных модулей, обеспечивающих централизованное накопление данных об образовательных результатах, их автоматизированный анализ и представление в виде аналитических отчетов.

Разрабатываемая система мониторинга успеваемости предназначена, с одной стороны, для предоставления преподавателям удобного инструмента анализа результатов контрольных мероприятий и динамики учебных достижений обучающихся, с другой стороны – для обеспечения административного уровня достоверной аналитической информацией, необходимой для принятия управленческих решений и повышения качества образовательного процесса.

Задачи, которые должна решать автоматизированная система:

- функция централизованного хранения результатов текущего и итогового контроля обучающихся;
- функция автоматического расчета показателей успеваемости (средний балл, процент выполнения, уровень освоения темы);

- функция анализа динамики успеваемости по обучающемуся, группе и учебному периоду;
- функция формирования аналитических отчетов в разрезе дисциплин, групп и отдельных обучающихся;
- функция визуализации данных (графики, диаграммы распределения, сравнительные показатели);
- функция разграничения прав доступа пользователей (администратор, преподаватель, обучающийся).

2.2 Ограничения и рекомендации к разработке системы

На разрабатываемую систему мониторинга успеваемости накладываются следующие ограничения и рекомендации:

- не рекомендуется вносить изменения в уже функционирующие в образовательной организации системы управления обучением или электронные журналы, поскольку данные системы внедрены в учебный процесс и используются на постоянной основе; их модификация может привести к нарушению стабильности работы, утрате части функционала и необходимости масштабного обновления программного обеспечения;
- не рекомендуется использовать альтернативную систему управления базами данных при наличии уже развернутой инфраструктуры хранения образовательных данных; переход на иную СУБД может потребовать переноса значительных объемов информации, дополнительной настройки серверного окружения и повлечь временные и организационные затраты;
- рекомендуется реализовать систему мониторинга успеваемости в виде веб-приложения, доступного через браузер на различных устройствах и операционных системах; это обеспечит универсальный доступ преподавателей и администрации к аналитической информации без установки специализированного клиентского программного обеспечения;
- рекомендуется использовать распространенный и поддерживаемый стек веб-технологий (например, JavaScript и сопутствующие фреймворки), что позволит обеспечить совместимость, упрощение сопровождения и возможность масштабирования системы;
- рекомендуется обеспечить соответствие системы требованиям информационной безопасности и защиты персональных данных

обучающихся, включая разграничение прав доступа и хранение данных в защищенном виде.

2.3 Требования к разработке системы

К разрабатываемой системе мониторинга успеваемости предъявляются следующие функциональные требования:

- централизованное хранения результатов текущего, промежуточного и итогового контроля обучающихся, а также данных о посещаемости занятий;
- проведение электронного тестирования с поддержкой различных типов заданий (выбор одного и нескольких вариантов, ввод текста, сопоставление, упорядочивание) и автоматической проверкой ответов;
- автоматический расчет показателей успеваемости (средний балл, процент посещаемости, результаты тестов, уровень освоения темы);
- расчет уровня освоения тем и формирование персональных рекомендаций по сложности последующих заданий;
- анализа динамики успеваемости по обучающемуся, группе и учебному периоду;
- формирование аналитических отчетов в разрезе дисциплин, групп и отдельных обучающихся с возможностью экспорта;
- визуализация данных (графики динамики оценок, диаграммы распределения, сравнительные показатели по группам);
- функция разграничения прав доступа пользователей (администратор, преподаватель, обучающийся).

К разрабатываемой системе мониторинга успеваемости предъявляется следующий набор нефункциональных требований:

- надежность, безопасность и отказоустойчивость: система не должна допускать утечки персональных данных обучающихся и сотрудников образовательной организации, должна обеспечивать стабильное функционирование в течение всего периода эксплуатации, а при возникновении сбоев – восстанавливать работоспособность в минимально возможные сроки;
- достоверность данных: информация о результатах учебной деятельности и аналитических расчетах должна быть корректной, актуальной и соответствовать исходным данным;

- масштабируемость: система должна обеспечивать возможность увеличения количества пользователей без существенного снижения производительности;
- переносимость и совместимость: система должна корректно функционировать на различных операционных системах и в современных веб-браузерах без установки дополнительного программного обеспечения;
- быстроедействие: время отклика системы при стандартной нагрузке не должно превышать нескольких секунд при стабильном интернет-соединении;
- удобство использования: интерфейс системы должен быть интуитивно понятным и обеспечивать быстрый доступ к основным функциям анализа и формирования отчетности.

2.4 Структура входной и выходной информации

Для обеспечения корректного функционирования разрабатываемой системы мониторинга успеваемости необходимо максимально полно и точно определить структуру входных и выходных данных. Четкая формализация данных позволяет: повысить надежность и устойчивость работы системы; определить состав информации, необходимой для реализации каждой функции; минимизировать избыточность и дублирование данных; обеспечить пользователей полной и актуальной выходной информацией.

Для реализации системы мониторинга успеваемости требуется следующий набор входной информации (таблица 2).

Таблица 2 – Набор входной информации системы

Информация	Источник информации	Описание
Учетные данные для входа	Все пользователи	Адрес электронной почты и пароль, отправленные через форму входа
Основные данные университета	Администратор	Кафедры, студенческие группы (с годом начала), предметы (с общим количеством часов и формой оценивания) и назначения преподавателей, связывающие преподавателя-предмет-группу-семестр-год
Учетные записи и профили	Администратор	Имя, контактная информация, должность), связанные с учетными записями пользователей
Записи о занятиях	Преподаватель	Отдельные занятия с типом (лекция/практика/лабораторная/семинар), временным интервалом, аудиторией и необязательным описанием темы.

Продолжение таблицы 2

Информация	Источник информации	Описание
Записи о посещаемости	Преподаватель	Отметки присутствия для каждого студента на дату занятия в рамках назначения преподавания, возможно с комментарием.
Вводимые оценки	Преподаватель	Значение и тип оценки (текущая/тематическая/за семестр/экзаменационная) для студента в рамках назначения, зафиксированные на определенную дату.
Контент банка вопросов	Преподаватель	Вопросы с текстом, необязательным изображением, уровнем сложности, типом вопроса (одиночный выбор, множественный выбор, текстовый ввод, сопоставление, упорядочивание), весом оценки и вариантами ответов.
Настройки тестов	Преподаватель	Метаданные теста: название, предмет, лимит времени, разрешенные попытки, процент проходного балла, флаги перемешивания, окно доступности и выбор вопросов.
Назначения тестов	Преподаватель	Назначение опубликованного теста группе или отдельному студенту, возможно с переопределением окна доступности.
Ответы студента	Студент	Полезные нагрузки ответов на каждый вопрос (ID выбранных вариантов, введенный текст, сопоставленные пары), отправленные во время активной сессии тестирования, вместе с затраченным временем.
Загруженные изображения	Преподаватель	Изображения PNG или JPEG, прикрепленные к телу вопроса.

Система мониторинга успеваемости в результате своей работы генерирует следующий набор выходной информации (таблица 3).

Таблица 3 – Набор выходной информации системы

Информация	Формат	Описание
Токен доступа	JSON	Подписанный токен носителя, выдаваемый при успешном входе, действителен в течение 60 минут и содержит ID пользователя и его роль.
Ответы сервера	JSON	Структурированные ответы для всех конечных точек ресурсов (списки, объекты с деталями, агрегированная статистика).
Автоматическая оценка за тест	Запись БД	Запись об оценке, создаваемая автоматически при завершении сессии. Если разрешено несколько попыток, сохраняется только лучший результат.

Продолжение таблицы 3

Информация	Формат	Описание
Адаптивная рекомендация	Запись БД	Запись для каждого студента и каждой темы, хранящая степень усвоения и рекомендуемую сложность, обновляемая с помощью экспоненциального скользящего среднего после каждой сессии.
Отчет об оценках (Excel)	XLSX файл	Форматированная электронная таблица с стилизованными заголовками, чередующейся заливкой строк и автоматически подогнанной шириной столбцов. Возможна фильтрация по группе, предмету, учебному году и типу оценки.
Отчет об оценках (PDF)	PDF файл	Документ формата А4, содержащий те же данные об оценках в виде форматированной таблицы с заголовком, чередующейся заливкой строк и повторяющимися заголовками столбцов на разных страницах.
Отчет о посещаемости (Excel)	XLSX файл	Записи посещаемости для каждого студента и каждого занятия с индикацией присутствия/отсутствия и необязательными комментариями. Возможна фильтрация по группе, предмету, диапазону дат.
Индивидуальный отчет студента (Excel)	XLSX файл	Многолистовая книга: Лист 1 - оценки по предметам; Лист 2 - показатели посещаемости; Лист 3 - результаты тестов с лучшим процентом и статусом сдачи/несдачи.
Индивидуальный отчет студента (PDF)	PDF файл	Отчет формата А4, объединяющий те же три раздела, что и в Excel версии, в единый печатный документ.
Агрегированный аналитический отчет (Excel)	XLSX файл	Трехлистовая книга: средние показатели по группам по предметам, рейтинг студентов по средней оценке и сводка по посещаемости.
Обслуживаемый статический медиа-контент	Файлы изображений (HTTP)	Загруженные изображения вопросов.

Графическое представление входной и выходной информации приведено на рисунке 11.

Структура входной и выходной информации Автоматизированной системы мониторинга успеваемости

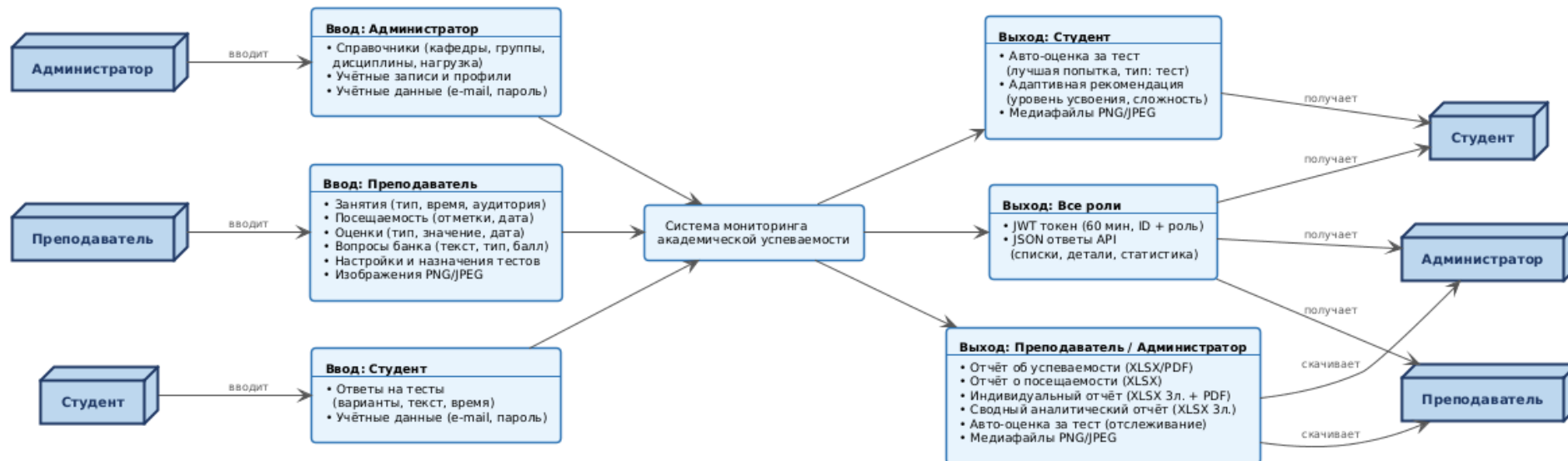


Рисунок 11 – Входная и выходная информация в системе

Помимо программной реализации системы, требуется разработка комплекта эксплуатационной документации, входящей в состав нормативно-справочной информации:

- руководство пользователя (преподавателя) – описание порядка внесения оценок, формирования отчетов, редактирования данных;
- руководство администратора системы – инструкции по настройке системы, управлению учетными записями пользователей, разграничению прав доступа, резервному копированию базы данных;
- краткое руководство для обучающихся (при наличии доступа) – описание порядка просмотра успеваемости и личных результатов.

Документация должна быть оформлена в структурированном и наглядном виде, с использованием пошаговых инструкций и иллюстративных материалов.

Таким образом, определение структуры входных и выходных данных, а также формализация функциональных требований создают основу для проектирования программной архитектуры системы и разработки алгоритмов ее функционирования. Вопросы проектирования и реализации системы рассматриваются в третьей главе работы.

3 Разработка проектных решений

3.1 Введение в разработку модели программной архитектуры

Прежде чем приступить к написанию кода, любая серьезная разработка требует проектирования: определения того, из каких частей будет состоять система, как они будут взаимодействовать и каким образом будут удовлетворяться требования пользователей. Именно этому посвящен этап анализа и проектирования программного обеспечения.

Применительно к разрабатываемой системе мониторинга успеваемости процесс проектирования преследовал три основные цели:

- сформировать системный проект, охватывающий все заявленные модули: тестирование, посещаемость, оценки, расписание, аналитику и экспорт отчетов;
- выстроить архитектуру, устойчивую к изменениям – такую, в которой добавление нового модуля не затрагивает уже работающие части системы;
- обеспечить адаптацию интерфейса и доступных функций под каждую из трех ролей пользователей: администратора, преподавателя и обучающегося.

На входе процесса проектирования находятся требования к системе и описание предметной области. На выходе – набор моделей: схема базы данных, диаграммы вариантов использования, компонентов и развертывания, которые в совокупности дают полное представление о будущей системе.

Следует отметить, что проектирование не является линейным процессом. В ходе работы неоднократно возникала необходимость вернуться к предыдущим решениям и уточнить их: например, изначально не было предусмотрено разделение оценок по типам, однако в процессе проработки модуля выставления оценок стало очевидно, что текущие, тематические, итоговые и тестовые оценки необходимо хранить отдельно. Подобные уточнения вносились итерационно и в конечном счете улучшили качество проектных решений.

Результирующая архитектура системы описывается через несколько видов UML-диаграмм, каждая из которых отвечает на свой вопрос: кто

пользуется системой и зачем, как устроена база данных, как взаимодействуют компоненты и где физически размещается система.

3.2 Моделирование системы с точки зрения прецедентов

Диаграмма вариантов использования (Use-Case) – это UML-диаграмма, которая отображает функциональность системы глазами ее пользователей. Она не описывает внутреннее устройство системы, а отвечает на вопрос: кто взаимодействует с системой и что именно он может в ней делать. Такая диаграмма удобна как для обсуждения требований с заказчиком, так и для последующей разработки – каждый вариант использования фактически соответствует функции, которую необходимо реализовать.

В разрабатываемой системе мониторинга успеваемости выделены три актора:

- администратор – занимается настройкой и поддержкой структуры системы: заводит пользователей, кафедры, учебные группы, предметы и назначает преподавателей на дисциплины;

- преподаватель – ведет текущую работу: отмечает посещаемость, выставляет оценки, составляет тесты, отслеживает успеваемость групп и выгружает отчеты;

- обучающийся – работает только со своими данными: смотрит оценки и посещаемость, проходит назначенные тесты и получает персональные рекомендации по обучению.

По результатам анализа были определены следующие варианты использования системы:

Общие для всех ролей: вход в систему; просмотр персональной панели управления; смена пароля.

Администратор: управление учетными записями; управление группами и кафедрами; управление предметами и преподавательской нагрузкой; управление расписанием занятий.

Преподаватель: ведение журнала посещаемости; выставление и редактирование оценок; формирование банка вопросов; создание, настройка и публикация тестов; назначение тестов группам и отдельным студентам;

просмотр аналитики успеваемости и тестирования; формирование отчетов и экспорт в XLSX / PDF.

Обучающийся: просмотр собственных оценок и посещаемости; прохождение тестов; просмотр результатов тестирования; просмотр адаптивных рекомендаций.

Графическое представление вариантов использования системы представлено на рисунке 12.

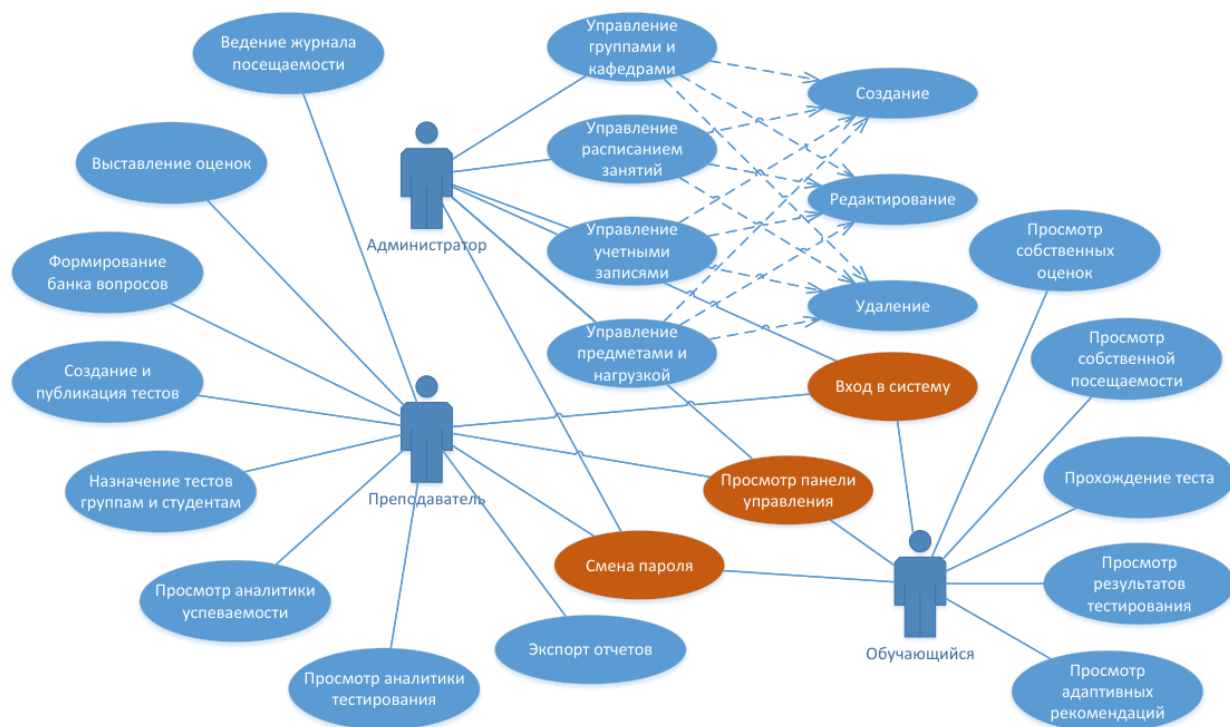


Рисунок 12 – Диаграмма вариантов использования

3.3 Моделирование системы с точки зрения последовательностей

Диаграмма последовательности UML – это диаграмма взаимодействия, которая отображает обмен сообщениями между объектами системы во времени. В отличие от диаграммы вариантов использования, которая описывает что система умеет делать, диаграмма последовательности раскрывает как именно это происходит: в каком порядке передаются сообщения, какие объекты в этом участвуют и что возвращается в ответ на каждый запрос. Это делает ее одним из наиболее практически ценных инструментов проектирования: по ней можно точно определить, какие методы

должны существовать у каждого объекта и какие данные необходимо передавать между уровнями системы.

Для разрабатываемой системы мониторинга успеваемости рассмотрены три ключевых сценария взаимодействия, охватывающих основные роли и функциональные модули. Первый сценарий – прохождение теста обучающимся (рисунок 13).

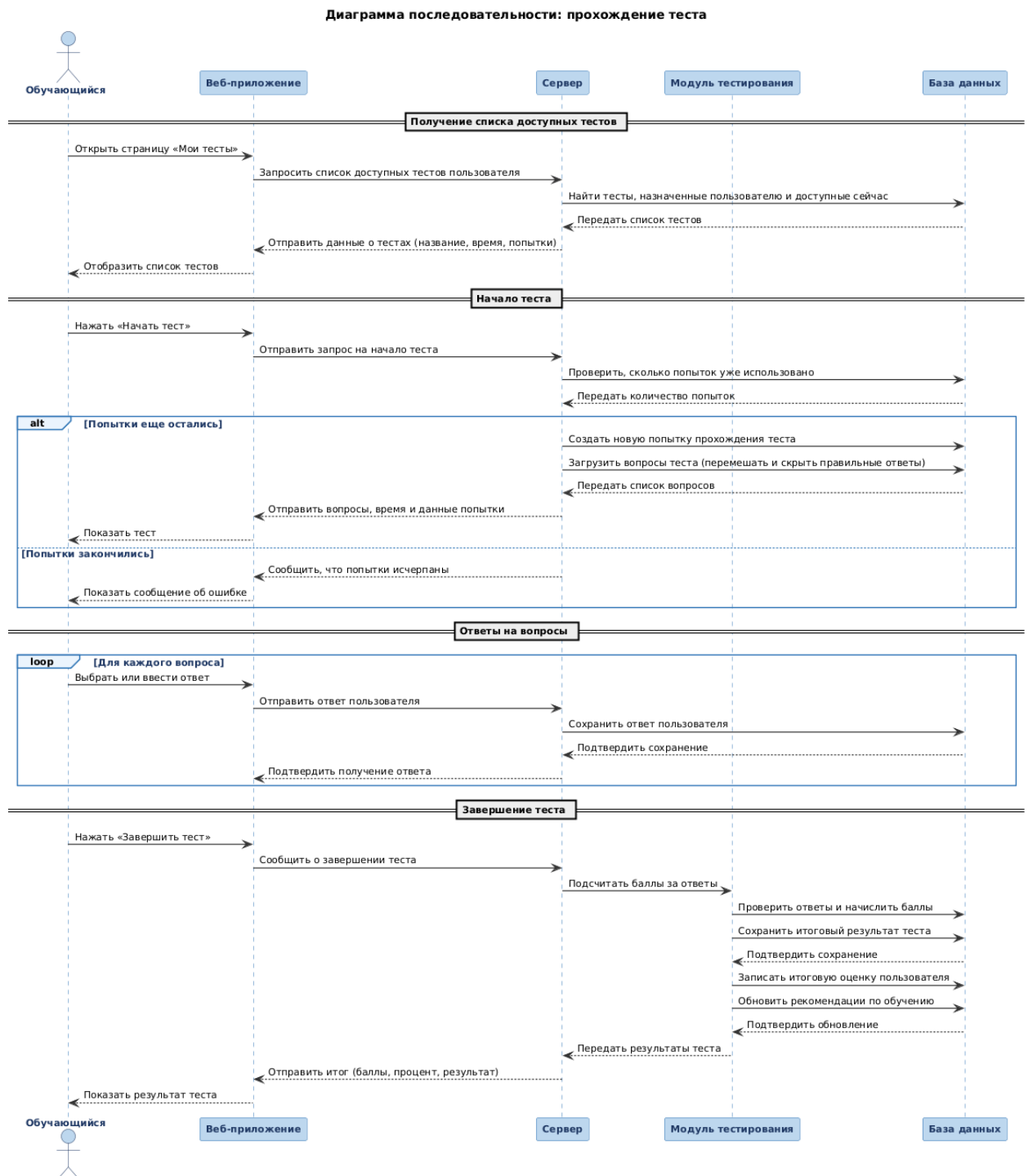


Рисунок 13 – Диаграмма последовательности прохождения теста

Сценарий начинается с того, что студент открывает страницу тестирования в клиентской части приложения. Клиент отправляет запрос к серверу для получения списка назначенных и доступных на текущий момент тестов. Сервер выполняет запрос к базе данных, проверяя таблицы на предмет актуальных назначений с учетом временного окна доступности. Полученный список возвращается клиенту в виде JSON-ответа.

После выбора теста студент инициирует начало сессии. Сервер проверяет, не превышен ли лимит попыток для данного студента и данного теста и, если проверка пройдена, создает новую запись в таблице со статусом «В процессе выполнения».

Затем сервер формирует список вопросов: при включенной опции перемешивания порядок вопросов меняется на стороне сервера, а не клиента – это исключает возможность определить правильный порядок путем анализа сетевых запросов.

Поля с правильными ответами из таблицы «Вопросы» при этом не передаются клиенту. Клиент отображает вопросы один за другим.

По мере ответа на вопросы каждый ответ студента отправляется на сервер и сохраняется в таблице «Ответы на вопросы». При завершении теста (либо по истечении времени, либо по нажатию кнопки завершения) сервер запускает процедуру оценивания: для каждого ответа вычисляется правильность и количество начисленных баллов с учетом типа вопроса и максимального балла.

После подсчета итогового результата запись сессии обновляется, автоматически создается оценка типа «тест» в таблице «Оценки», а уровень освоения каждой темы, затронутой в тесте, пересчитывается по формуле экспоненциального скользящего среднего и сохраняется в таблице рекомендаций.

Второй сценарий – выставление посещаемости преподавателем (рис. 14).

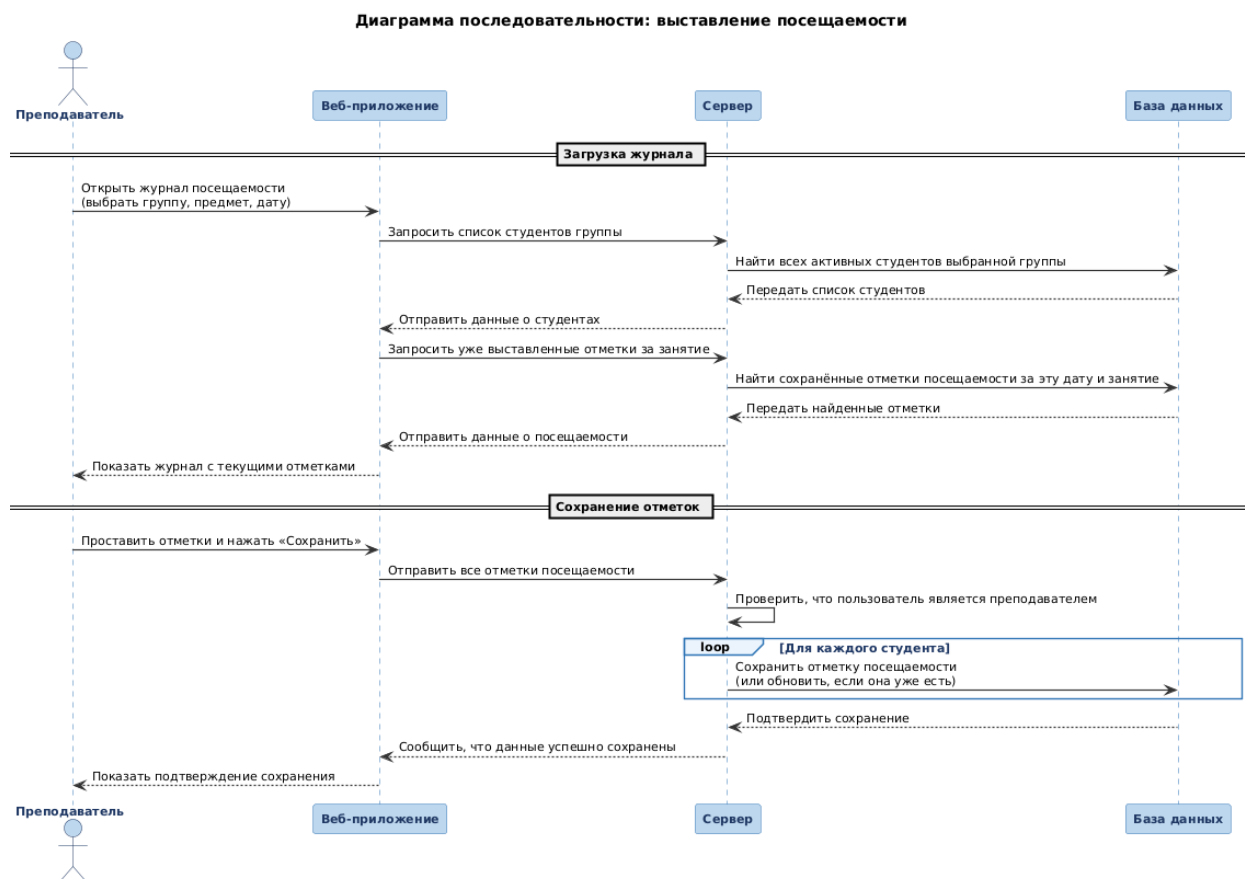


Рисунок 14 – Диаграмма последовательности выставления посещаемости

Преподаватель открывает журнал посещаемости и выбирает группу, предмет и дату занятия. Клиентская часть направляет на сервер запрос на получение списка студентов выбранной группы, а также уже сохраненных отметок за указанную дату (если они существуют).

Сервер выполняет два последовательных запроса к базе данных: первый – к таблице «Студенты» с фильтрацией по группе, второй – к таблице «Посещаемость» с фильтрацией по назначению группы преподавателю и дате занятия.

Полученные данные объединяются и передаются клиенту, который отображает готовый журнал с предзаполненными значениями там, где отметки уже были поставлены ранее.

После того как преподаватель проставил все отметки, он отправляет их на сервер единым пакетным запросом. Для каждой записи сервер выполняет операцию обновления данных с обработкой конфликтов: если запись для данной тройки (студент, назначение группы, дата занятия) уже существует, она обновляется, что позволяет редактировать посещаемость задним числом

без риска дублирования данных. Уникальность записи гарантируется ограничением на уровне базы данных.

Третий сценарий – формирование аналитики (рисунок 15).

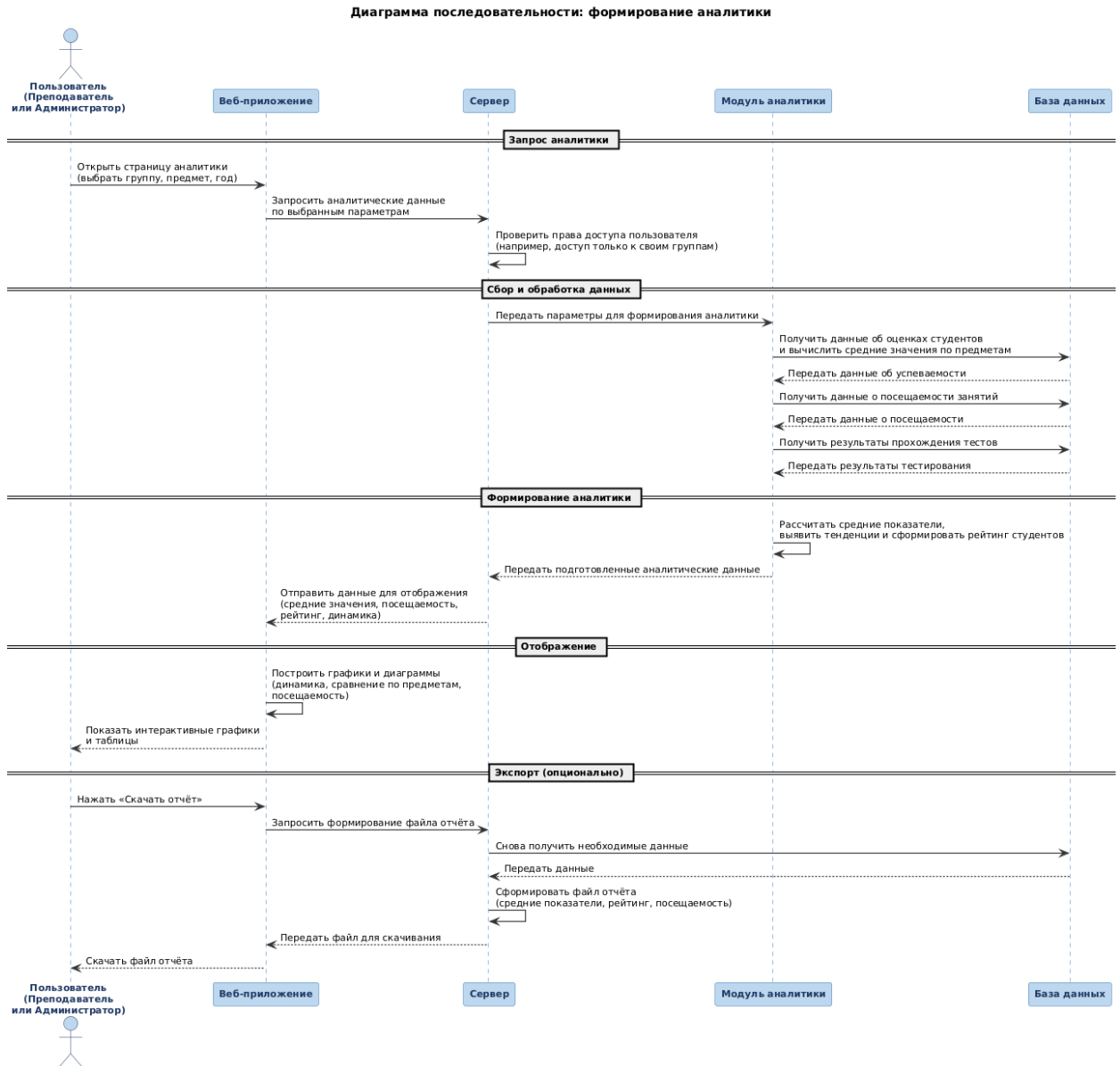


Рисунок 15 – Диаграмма последовательности выставления посещаемости

Пользователь (преподаватель или администратор) открывает страницу аналитики, выбирает параметры: группу, предмет и учебный год. Клиентская часть отправляет запрос к аналитическому модулю на сервере. Модуль аналитики прежде всего проверяет область данных пользователя: для преподавателей, не являющихся администраторами, ограничивает выборку только теми группами и предметами, на которые данный преподаватель назначен. Это исключает возможность получить данные чужих групп даже при прямом обращении к API.

Затем модуль выполняет серию агрегирующих SQL-запросов к базе данных: вычисляет средние оценки по предметам, процент посещаемости, результаты тестирования, рейтинг студентов и динамику успеваемости по семестрам.

Все вычисления производятся непосредственно на уровне СУБД с применением агрегатных функций (AVG, COUNT, SUM) и оконных функций, что исключает необходимость загружать сырые данные в память сервера и обрабатывать их средствами Python. Полученный структурированный JSON-ответ передается клиенту, который отображает результаты в виде интерактивных графиков с использованием библиотеки Recharts.

При необходимости пользователь может экспортировать аналитику в файл Excel или PDF, инициировав соответствующий запрос к модулю отчетности.

Таким образом, рассмотренные диаграммы последовательностей наглядно демонстрируют, каким образом клиентская часть, серверные модули и база данных взаимодействуют при выполнении ключевых операций системы, и позволяют убедиться в том, что архитектурные решения – асинхронное API, ролевые проверки, серверное перемешивание вопросов, пакетное сохранение посещаемости – реализованы последовательно и без избыточных зависимостей между компонентами.

3.4 Моделирование системы с точки зрения проектирования сущностей

Диаграмма классов UML – это структурная диаграмма, которая описывает состав системы на уровне её сущностей: какие объекты в ней существуют, какими свойствами они обладают и в каких отношениях находятся между собой.

В отличие от диаграммы последовательности, которая описывает динамику поведения системы во времени, диаграмма классов носит статический характер и отражает её структуру в любой момент времени. Она является одной из ключевых диаграмм при проектировании реляционных баз данных: каждый класс соответствует таблице, каждый атрибут – столбцу, а связи между классами – внешним ключам и ограничениям целостности.

В разрабатываемой системе мониторинга академической успеваемости

каждый класс непосредственно будет отражать таблицу базы данных PostgreSQL, реализованную в виде ORM-модели SQLAlchemy. Это означает, что диаграмма классов одновременно будет служить и схемой базы данных, и проектной моделью серверной части приложения.

Центральной сущностью системы является класс User. Он хранит аутентификационные данные пользователя: уникальный адрес электронной почты, хэш пароля (алгоритм bcrypt), роль (перечисление UserRole со значениями admin, teacher, student), признак активности учётной записи и временные метки создания, обновления и последнего входа. Для глобальной идентификации записи предусмотрено поле uuid типа UUID, генерируемое автоматически.

Класс User связан отношением «один к одному» с классами Teacher и Student: первый содержит академический профиль преподавателя (ФИО, должность, телефон, привязка к кафедре), второй – профиль студента (ФИО, дата рождения, номер студенческого билета, привязка к учебной группе, телефон). Разделение аутентификационных и профильных данных позволяет независимо управлять учётными записями и академическими сведениями, а также легко добавлять новые типы профилей в будущем.

Учебная структура организована иерархически. Корневым элементом служит класс Department (кафедра), который хранит наименование, краткий код и описание подразделения. С кафедрой связаны три зависимые сущности: Group (учебная группа), Subject (учебная дисциплина) и Teacher (преподаватель).

Класс Group описывает академическую группу: её наименование и год начала обучения; каждая группа включает одного или нескольких студентов (Student).

Класс Subject описывает учебную дисциплину с указанием наименования, кода дисциплины и общего количества учебных часов. Внутри каждой дисциплины определяется иерархия тем с помощью класса Topic: темы могут быть вложены друг в друга через ссылку на родительскую тему (поле parent_id, самосвязь), что позволяет строить дерево тематических разделов произвольной глубины. Именно темы являются наименьшей единицей отслеживания прогресса в адаптивной подсистеме.

Ключевой связующей сущностью всей системы является класс TeachingAssignment – назначение конкретного преподавателя на конкретный предмет в конкретной группе в рамках заданного семестра и учебного года.

Уникальность комбинации (teacher_id, subject_id, group_id, semester, acad_year) защищена составным ограничением UNIQUE, что исключает возможность дублирования назначений. Все оценки, записи посещаемости и занятия в системе привязаны именно к записи TeachingAssignment, а не напрямую к преподавателю или предмету. Такое архитектурное решение позволяет однозначно разграничить данные разных семестров, а также корректно обработать ситуацию, когда один и тот же предмет в разные годы ведут разные преподаватели.

Класс Lesson описывает отдельное занятие в рамках назначения: его тип (перечисление LessonType: лекция, практика, лабораторная, семинар, прочее), время начала и окончания, аудиторию и тему занятия. Класс Grade хранит оценку студента: её числовое значение, тип (перечисление GradeType: current – текущая, thematic – тематическая, midterm – рубежная, final – итоговая, attendance – за посещаемость, test – тестовая), признак прохождения (passed), дату выставления и ссылку на преподавателя, внесшего оценку. Оценки типа test создаются системой автоматически по завершении тестовой сессии и содержат ссылку на соответствующую запись TestSession через поле session_id. Класс Attendance фиксирует факт посещения студентом конкретного занятия: хранит дату занятия, признак присутствия (is_present), необязательный комментарий и ссылку на записавшего преподавателя.

Уникальность записи по тройке (student_id, assignment_id, lesson_date) гарантируется ограничением базы данных, что исключает дублирование отметок.

Подсистема тестирования представлена взаимосвязанной цепочкой из шести классов. Класс Test описывает тест в целом: заголовок, описание, статус (перечисление TestStatus: draft – черновик, published – опубликован, archived – архив), лимит времени в минутах, допустимое количество попыток, проходной балл в процентах, признаки перемешивания вопросов и вариантов ответа, признак отображения результатов после прохождения, а также временное окно доступности (available_from, available_to). Класс Question реализует банк вопросов и поддерживает пять типов заданий: single_choice (одиночный выбор из предложенных вариантов), multiple_choice

(множественный выбор), `text_input` (ввод текстового ответа), `matching` (сопоставление пар) и `ordering` (расстановка элементов в правильном порядке). Варианты ответов и сопоставления хранятся в поле `options` типа JSONB, что обеспечивает гибкость хранения структурно разнородных данных без изменения схемы таблицы.

К вопросу может быть прикреплено изображение (поле `image_url`). Класс `TestQuestion` является соединительной таблицей между `Test` и `Question`: благодаря ей один и тот же вопрос может входить в состав нескольких тестов, что исключает дублирование содержимого банка вопросов.

Класс `TestAssignment` определяет, какой группе или конкретному студенту назначен тест, и при необходимости позволяет переопределить временное окно доступности для отдельного студента. Каждое прохождение теста фиксируется в классе `TestSession`: хранятся порядок вопросов (в виде JSONB-массива идентификаторов), статус сессии (`in_progress`, `completed`, `timed_out`, `abandoned`), итоговый балл и максимально возможный балл, признак успешного прохождения и IP-адрес клиента. Ответы студента на каждый вопрос сессии хранятся в классе `QuestionAnswer`: помимо данных ответа (`answer_data`, JSONB), фиксируются набранный балл, признак правильности и время, затраченное на ответ (`time_spent_sec`).

Адаптивная подсистема рекомендаций реализована классом `AdaptiveRecommendation`. Для каждой пары «студент – тема» хранится уровень освоения (`mastery_level`, числовое значение от 0 до 1, контролируемое ограничением CHECK) и рекомендуемая сложность следующего задания (поле `recommended_difficulty` типа `Difficulty`: `easy`, `medium` или `hard`). После завершения каждой тестовой сессии значение `mastery_level` пересчитывается по формуле экспоненциального скользящего среднего (1):

$$mastery_new = \alpha \cdot score_pct + (1 - \alpha) \cdot mastery_old \quad (1)$$

где α – коэффициент сглаживания, равный 0,4;

`score_pct` – процент верного прохождения теста в десятичном представлении;

`mastery_old` – предыдущее значение уровня освоения.

Такой подход позволяет учитывать как текущий результат студента, так и его предыдущую историю, постепенно корректируя рекомендацию без резких скачков при случайно высоком или низком результате.

Полная диаграмма классов системы представлена на рисунке 16.

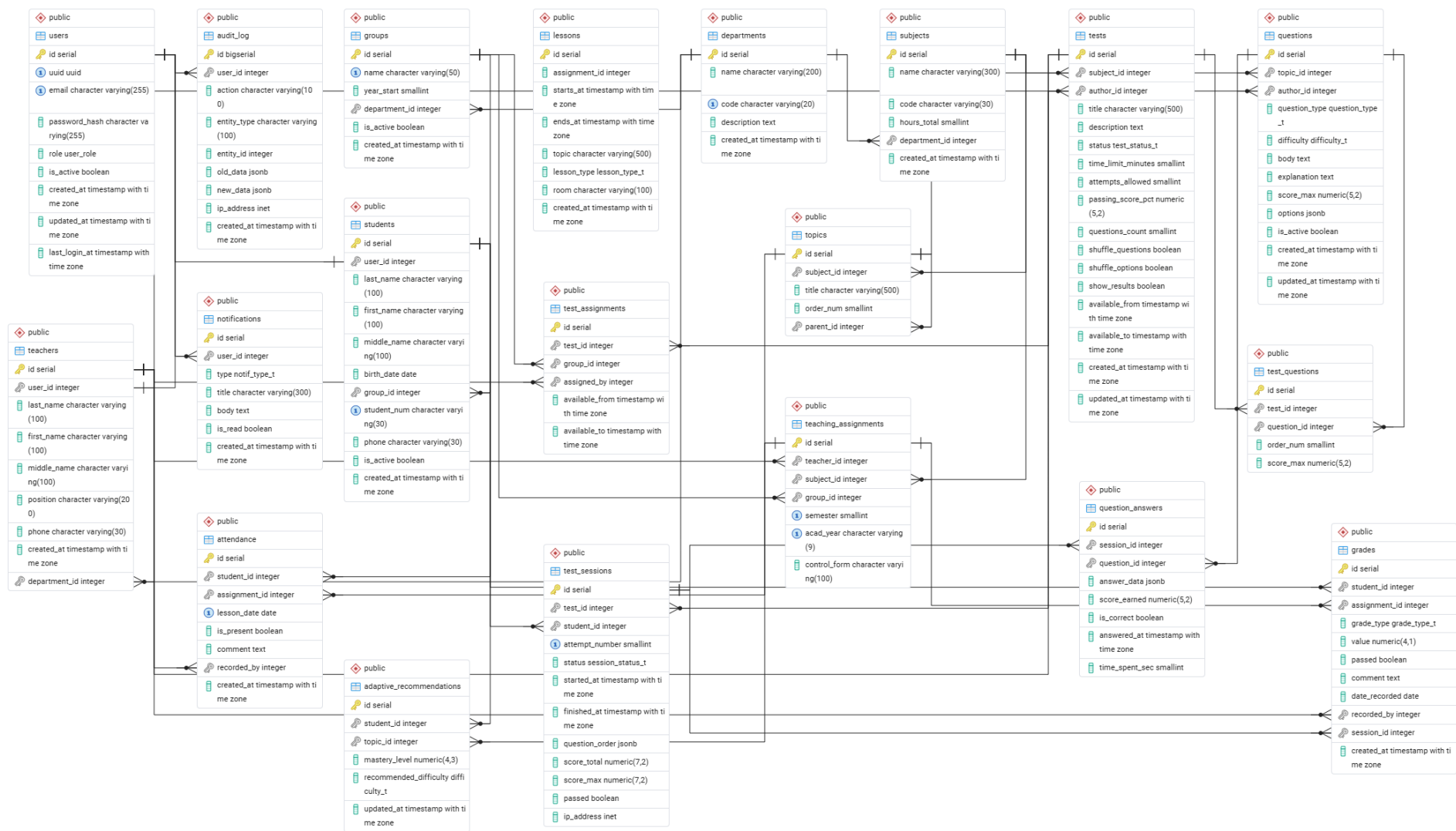


Рисунок 16 – ER-диаграмма системы мониторинга успеваемости

3.5 Моделирование системы с точки зрения компонентов

Диаграмма компонентов UML предназначена для описания системы в терминах её составных блоков: модулей, библиотек, исполняемых файлов и баз данных. Каждый компонент инкапсулирует определённую функциональность и взаимодействует с другими через чётко определённые интерфейсы. Диаграмма компонентов позволяет разработчикам наглядно понять структуру системы, определить границы ответственности каждого блока и выявить зависимости между ними ещё на этапе проектирования, что снижает риск возникновения архитектурных проблем в ходе реализации.

Разрабатываемая система имеет двухуровневую клиент-серверную архитектуру. Клиентская часть выполняется непосредственно в браузере пользователя, серверная – на сервере приложений. Взаимодействие между ними осуществляется по протоколу HTTPS посредством REST API: клиент отправляет HTTP-запросы, сервер формирует и возвращает ответы в формате JSON.

Файлы экспортируемых отчётов (XLSX, PDF) передаются клиенту в режиме потоковой передачи непосредственно из буфера в памяти сервера, без создания временных файлов на диске. Запросы к статическим медиафайлам (изображения вопросов) обрабатываются отдельным статическим маршрутом /media.

Клиентская часть реализована в виде одностраничного приложения (SPA) на React с TypeScript, собираемого сборщиком Vite. Приложение структурировано в виде нескольких взаимосвязанных компонентов. Компонент маршрутизации на основе React Router сопоставляет URL-адреса с соответствующими страницами; все маршруты, требующие авторизации, обернуты в компонент ProtectedRoute, который при отсутствии JWT-токена в хранилище немедленно перенаправляет пользователя на страницу входа.

Компонент Layout динамически формирует боковое меню навигации в зависимости от роли авторизованного пользователя: администратору доступны инструменты управления структурой системы, преподавателю – рабочие инструменты ведения журнала и тестирования, обучающемуся – личный кабинет с оценками, посещаемостью и тестами.

Глобальное состояние сессии управляется через хранилище Zustand (authStore). При первом запуске приложения хранилище выполняет запрос GET /api/auth/me для восстановления сессии по сохранённому токenu, что

позволяет пользователю оставаться авторизованным после перезагрузки страницы. Все HTTP-запросы к API выполняются через единственный сконфигурированный экземпляр клиента Axios с базовым URL сервера. Его перехватчик запросов автоматически добавляет заголовок Authorization: Bearer <token> ко всем исходящим запросам; перехватчик ответов при получении кода 401 очищает хранилище и перенаправляет на страницу входа без вмешательства со стороны отдельных страниц.

Графики и диаграммы аналитических данных отображаются с помощью библиотеки Recharts; все элементы пользовательского интерфейса реализованы на компонентах Material UI 9 с единой темой оформления.

Серверная часть построена на асинхронном фреймворке FastAPI и организована в виде независимых модулей-роутеров. Каждый роутер регистрируется в главном приложении (main.py) с собственным префиксом URL, что обеспечивает предсказуемую и читаемую структуру API. Все маршруты защищены механизмом dependency injection: зависимость get_current_user проверяет JWT-токен при каждом запросе, а ролевые зависимости require_admin, require_teacher и require_student применяются на уровне конкретного маршрута и отклоняют неавторизованные вызовы с кодом HTTP 403 до выполнения каких-либо операций с базой данных. Система включает следующие серверные модули:

- модуль авторизации и управления пользователями (/api/auth, /api/users, /api/teachers, /api/students) – обеспечивает аутентификацию пользователей (проверку пароля, выдачу JWT-токена, получение профиля текущего пользователя), управление учётными записями и профилями преподавателей и студентов. Пароли хранятся в виде bcrypt-хэшей с автоматически генерируемой солью, токены подписываются алгоритмом HS256 и имеют ограниченный срок действия;

- модуль учебной структуры (/api/departments, /api/groups, /api/subjects, /api/topics, /api/teaching-assignments) – объединяет операции создания и управления кафедрами, учебными группами, дисциплинами, темами и назначениями преподавателей. Этот модуль является фундаментом системы: большинство остальных модулей работают с данными, привязанными к записям TeachingAssignment, создаваемым здесь;

– модуль расписания (/api/lessons) – управляет занятиями в рамках назначений: позволяет создавать события с указанием типа, временного интервала, аудитории и темы, а также получать расписание занятий для группы или преподавателя;

– модуль посещаемости (/api/attendance) – принимает пакетные записи о присутствии студентов и сохраняет их с контролем уникальности на уровне базы данных (ON CONFLICT DO UPDATE). Поддерживает получение журнала посещаемости по группе, предмету и дате;

– модуль оценок (/api/grades) – обеспечивает ввод, обновление и удаление оценок преподавателем, а также принимает автоматически созданные тестовые оценки от модуля тестирования. Поддерживает фильтрацию по студенту, предмету, типу оценки и периоду;

– модуль тестирования (/api/tests, /api/questions, /api/sessions, /api/upload) – реализует полный жизненный цикл теста: создание и редактирование банка вопросов с поддержкой пяти типов и прикрепления изображений, настройку и публикацию теста, назначение теста группам и отдельным студентам, управление сессиями (старт, сохранение ответов, завершение по кнопке или по таймауту), автоматическую проверку ответов, подсчёт баллов и ограничение числа попыток;

– адаптивный модуль (интегрирован в /api/sessions) – активируется автоматически при завершении сессии. Определяет, какие темы встретились в тесте, вычисляет новое значение mastery_level по формуле EMA и обновляет рекомендуемую сложность в таблице adaptive_recommendations;

– модуль аналитики (/api/analytics) – выполняет агрегирующие SQL-запросы и предоставляет сводные данные: средние оценки по предметам, рейтинг студентов, динамику успеваемости по семестрам, статистику ошибок по вопросам и сравнение групп. Для преподавателей, не являющихся администраторами, область данных автоматически ограничивается их назначениями через вспомогательную функцию _teacher_scope();

– модуль отчётности (/api/reports) – формирует и передаёт клиенту файлы отчётов в форматах XLSX (библиотека openpyxl: многолистовые книги с форматированием, автоматической шириной столбцов и чередующейся заливкой строк) и PDF (библиотека ReportLab: документы формата A4 с

повторяющимися заголовками таблиц на каждой странице). Все файлы формируются из буфера и передаются клиенту как потоковый ответ;

– модуль статических файлов (/media) – обеспечивает загрузку изображений для вопросов через multipart-форму и их последующую раздачу по HTTP-маршруту; URL изображения сохраняется в поле `image_url` таблицы `questions`.

В основе всей серверной части находится СУБД PostgreSQL версии 15 и выше. Взаимодействие с базой данных осуществляется асинхронно через библиотеку SQLAlchemy 2.0 с низкоуровневым драйвером `asyncpg`. Такая связка обеспечивает неблокирующее выполнение запросов в рамках `event loop` Python, что позволяет серверу обрабатывать множество одновременных HTTP-запросов без создания отдельных потоков. Параметры подключения к базе данных, секретный ключ подписи токенов и срок действия токена вынесены в файл переменных окружения и читаются при старте.

Диаграмма компонентов системы представлена на рисунке 17.

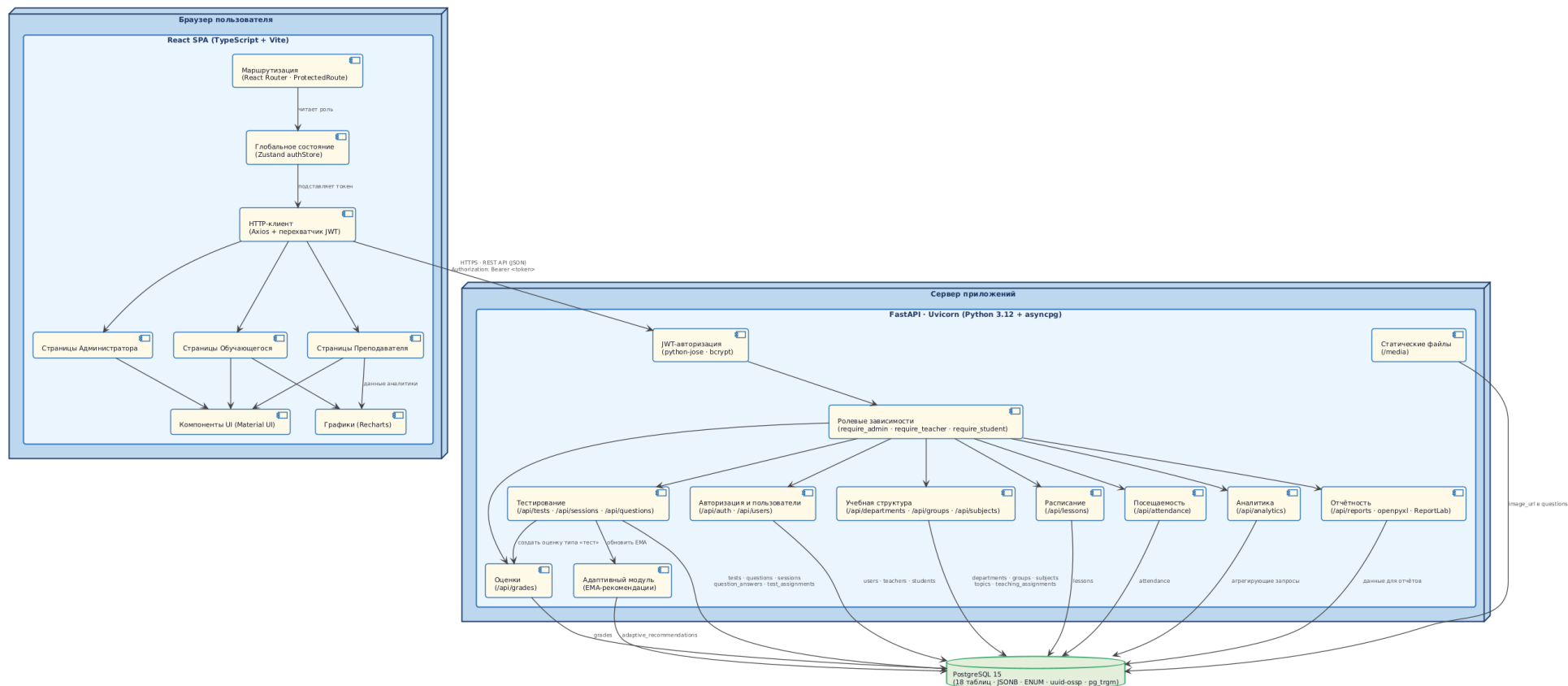


Рисунок 17 – UML-диаграмма компонентов системы мониторинга успеваемости

3.6 Моделирование системы с точки зрения развертывания

Диаграмма развёртывания UML описывает физическое размещение компонентов системы: на каком оборудовании или в какой среде исполнения они работают, как связаны между собой и по каким протоколам обмениваются данными. В отличие от диаграммы компонентов, которая отвечает на вопрос «из чего состоит система», диаграмма развёртывания отвечает на вопрос «где именно каждая её часть работает». Это делает её незаменимым инструментом при планировании инфраструктуры, настройке сетевого взаимодействия и документировании готового решения.

Разрабатываемая система развёртывается в составе трёх узлов: клиентского устройства пользователя, сервера приложений и сервера базы данных. В простейшей конфигурации (локальное развёртывание) сервер приложений и сервер базы данных могут располагаться на одной физической машине; в производственной среде их принято разносить на отдельные узлы для обеспечения изоляции и масштабируемости.

Клиентский узел представлен браузером пользователя на любом устройстве (персональный компьютер, ноутбук, планшет). Поскольку клиентская часть реализована в виде SPA на React, при первом обращении браузер загружает статические файлы приложения (JavaScript-бандл, CSS, HTML-оболочку), после чего все последующие запросы к данным выполняются в фоновом режиме через REST API без полной перезагрузки страницы. Связь с сервером приложений осуществляется по протоколу HTTPS на порту 443.

Сервер приложений является центральным узлом системы. На нём развёртываются два компонента. Первый – React SPA в собранном виде (статические файлы, сгенерированные командой `npm run build` с помощью сборщика Vite): они раздаются клиенту либо встроенными средствами FastAPI, либо отдельным веб-сервером (например, Nginx). Второй компонент – серверное приложение FastAPI, запускаемое через ASGI-сервер Uvicorn. Uvicorn обрабатывает входящие HTTP-соединения, передаёт запросы фреймворку FastAPI и возвращает сформированные ответы клиенту. Помимо API-маршрутов, FastAPI обеспечивает раздачу загружаемых медиафайлов через статический маршрут `/media`. Конфигурация сервера (порт подключения к базе данных, секретный ключ, срок действия токена) хранится в файле `.env`, который не включается в репозиторий исходного кода.

В процессе разработки проксирование запросов от клиента к серверу API выполняется средствами Vite Dev Server: в конфигурационном файле vite.config.ts определено, что запросы с путями /api и /media перенаправляются на адрес http://localhost:8000, где запущен Uvicorn. Это позволяет избежать проблем с политикой CORS при локальной разработке.

Узел базы данных обеспечивает хранение и обработку всех данных системы. На нём развёртывается СУБД PostgreSQL версии 15 и выше, прослушивающая стандартный порт 5432. Взаимодействие между сервером приложений и сервером базы данных происходит по внутренней сети через асинхронный драйвер. База данных содержит 18 таблиц, 7 пользовательских типов-перечислений (ENUM), расширения uuid-ossr (для генерации UUID) и pg_trgm (для полнотекстового поиска по именам студентов и преподавателей), а также набор индексов для ускорения часто выполняемых аналитических запросов.

Таким образом, разработанная архитектура развёртывания обеспечивает чёткое разделение ответственности между узлами системы: клиентский узел отвечает исключительно за отображение интерфейса, сервер приложений – за обработку бизнес-логики и формирование API-ответов, а узел базы данных – за надёжное хранение и транзакционную обработку данных.

Диаграмма развёртывания системы представлена на рисунок 18.

4 Выбор программного и технического обеспечения и его обоснование

4.1 Выбор и обоснование выбора программного обеспечения

Разработка современных веб-приложений предполагает обоснованный выбор технологического стека: языков программирования, фреймворков, библиотек и инструментов сборки. От этого выбора напрямую зависят производительность системы, скорость разработки, удобство сопровождения и возможность дальнейшего масштабирования. В настоящее время на рынке представлено значительное число конкурирующих решений для каждого уровня веб-приложения, поэтому перед разработчиком всегда стоит задача сравнительного анализа и обоснованного выбора.

Рассмотрим основные технологии, применявшиеся при разработке системы мониторинга академической успеваемости, а также альтернативы, которые рассматривались в ходе проектирования.

Для реализации серверной части приложения рассматривались два наиболее распространённых в области веб-разработки языка: Python и Java.

Python – это высокоуровневый интерпретируемый язык программирования общего назначения с динамической типизацией и лаконичным синтаксисом. Он широко применяется в веб-разработке, анализе данных, машинном обучении и автоматизации. Python обладает богатой экосистемой библиотек и фреймворков, активным сообществом разработчиков и подробной документацией. Для решения задач асинхронного ввода-вывода в Python начиная с версии 3.4 доступна стандартная библиотека `asyncio`, на основе которой построены современные асинхронные фреймворки.

Java – это строго типизированный объектно-ориентированный язык программирования, работающий на виртуальной машине JVM. Java обеспечивает высокую производительность при работе с многопоточными приложениями, хорошо масштабируется и широко используется в корпоративной разработке. Однако по сравнению с Python Java требует значительно большего объёма шаблонного кода, что замедляет скорость разработки.

Основные различия между Python и Java применительно к задачам разработки системы мониторинга представлены в таблице 4.

Таблица 4 – Сравнение языков программирования в контексте использования их в разработке системы

Критерий	Python	Java
Синтаксис	Лаконичный, требует меньше строк кода	Более многословный, требует больше кода
Скорость разработки	Быстрая разработка прототипов и бизнес-логики	Ориентация на долгосрочные корпоративные проекты
Обработка параллелизма	Асинхронная модель (asyncio + asynprg), эффективна в одном потоке	Многопоточность, выше потребление памяти
Работа с подключениями БД	Эффективна при большом количестве одновременных запросов	Аналог достигается сложнее и затратнее по ресурсам
Экосистема и библиотеки	Готовые решения: FastAPI, SQLAlchemy, openpyxl, ReportLab, python-jose	Требуется подбор и настройка зависимостей, более сложная конфигурация

Таким образом, для разработки серверной части выбран язык Python как обеспечивающий высокую скорость разработки, нативную поддержку асинхронного программирования и наличие всех необходимых библиотек в экосистеме.

Для реализации REST API рассматривались два фреймворка на языке Python: FastAPI и Django REST Framework (DRF).

FastAPI – это современный асинхронный веб-фреймворк для Python, основанный на стандарте ASGI и построенный поверх библиотеки Starlette. Он обеспечивает автоматическую генерацию документации OpenAPI, встроенную валидацию запросов и ответов через Pydantic, а также нативную поддержку асинхронных обработчиков через async/await. FastAPI стабильно входит в число самых быстрых Python-фреймворков по результатам независимых бенчмарков.

Django REST Framework (DRF) – это надстройка над фреймворком Django, предоставляющая инструменты для построения RESTful API. DRF обладает богатым набором встроенных возможностей: сериализаторы, аутентификация, разграничение прав. Однако Django изначально проектировался как синхронный фреймворк; поддержка асинхронности была добавлена позднее и не является его основным режимом работы.

Основные различия между FastAPI и DRF применительно к задачам разработки системы мониторинга представлены в таблице 5.

Таблица 5 – Сравнение фреймворков реализации REST API в контексте использования их в разработке системы

Критерий	FastAPI	DRF
Модель выполнения	Полностью асинхронный по умолчанию, неблокирующая обработка запросов	Преимущественно синхронный режим
Работа с данными	Использует Pydantic: автоматическая валидация и сериализация	Требуются отдельные классы-сериализаторы
Документация API	Автоматическая генерация Swagger UI (/docs)	Требуются дополнительные пакеты
Внедрение зависимостей и авторизация	Гибкий dependency injection, удобно для ролевых проверок	Используется система permissions, менее гибкая в сложных сценариях

На основании проведённого сравнения для разработки выбран FastAPI, поскольку он обеспечивает нативную асинхронность, минимальный объём шаблонного кода и автоматическую документацию, что в совокупности ускоряет разработку и снижает вероятность ошибок.

В качестве ASGI-сервера для запуска FastAPI-приложения используется Uvicorn – высокопроизводительный асинхронный HTTP-сервер, официально рекомендованный авторами FastAPI.

Для взаимодействия с базой данных рассматривались SQLAlchemy и Tortoise ORM.

SQLAlchemy – это наиболее зрелая и функционально полная ORM-библиотека для Python. Начиная с версии 2.0 она поддерживает полностью асинхронный режим работы через интерфейс AsyncSession и AsyncEngine. SQLAlchemy предоставляет как декларативный стиль описания моделей (Mapped / mapped_column), так и мощный конструктор запросов select() для построения сложных агрегирующих запросов.

Tortoise ORM – более молодая асинхронная ORM, вдохновлённая Django ORM и изначально проектировавшаяся для async-окружений. Tortoise ORM имеет более простой синтаксис, однако уступает SQLAlchemy по зрелости, документации и поддержке сложных запросов с оконными функциями и агрегацией.

Для разработки выбрана SQLAlchemy в связке с асинхронным драйвером asyncpg – данная комбинация обеспечивает наилучшую производительность при работе с PostgreSQL в асинхронном режиме и поддерживает все специфические типы данных СУБД.

В качестве СУБД рассматривались PostgreSQL и MySQL.

PostgreSQL – это объектно-реляционная СУБД с открытым исходным кодом, известная своей надёжностью, соответствием стандартам SQL и богатым набором типов данных. PostgreSQL поддерживает JSONB (бинарное хранение JSON с возможностью индексирования), пользовательские типы-перечисления (ENUM), оконные функции, полнотекстовый поиск с индексами GIN, расширение uuid-ossr для генерации UUID и pg_trgm для нечёткого поиска по строкам. PostgreSQL обеспечивает высокую целостность данных через строгое следование принципам ACID.

MySQL – это широко распространённая реляционная СУБД с открытым исходным кодом. MySQL проще в начальной настройке и хорошо зарекомендовала себя в простых веб-приложениях. Однако по сравнению с PostgreSQL она обладает ограниченной поддержкой стандарта SQL, не имеет нативного типа JSONB с полноценным индексированием и менее развитыми механизмами работы с перечислениями и оконными функциями.

Основные различия между PostgreSQL и MySQL применительно к задачам разрабатываемой системы приведены в таблице 6.

Таблица 6 – Сравнение СУБД в контексте использования их в разработке системы

Критерий	PostgreSQL	MySQL
Работа с JSON	Поддержка JSONB и GIN-индексы для эффективных запросов	Поддержка JSON без нативного GIN-индексирования
ENUM-типы	Нативная поддержка с высокой целостностью данных	Поддержка есть, но менее гибкая при миграциях
Аналитические возможности	Развитые оконные функции и агрегаты	Функциональность менее полная
Поиск по частичному совпадению	Расширение pg_trgm для эффективного поиска	Ограниченные возможности реализации

На основании проведённого анализа для разработки выбрана СУБД PostgreSQL, как наиболее соответствующая требованиям разрабатываемой системы по набору поддерживаемых типов данных, производительности агрегирующих запросов и целостности данных.

Для реализации клиентской части рассматривались два подхода: разработка SPA на React с TypeScript и на Vue.js с JavaScript.

React – это библиотека для построения пользовательских интерфейсов, использующая компонентный подход: интерфейс представляет собой дерево переиспользуемых компонентов, каждый из которых управляет собственным состоянием. React поддерживает виртуальный DOM, что обеспечивает эффективное обновление интерфейса без полной перерисовки страницы. Начиная с версии 19, React включает улучшенную поддержку конкурентного рендеринга.

Vue.js – это фреймворк для построения пользовательских интерфейсов, отличающийся мягкой кривой обучения, однофайловыми компонентами и реактивной системой привязки данных. Однако экосистема Vue.js несколько меньше, чем у React, а выбор компонентных библиотек и инструментов управления состоянием ограничен по сравнению с React.

Для разработки выбран React, поскольку он обладает наибольшей экосистемой среди клиентских фреймворков, активно поддерживается и располагает всеми необходимыми библиотеками для реализации требуемого функционала.

В сочетании с React используется TypeScript – строго типизированная надстройка над JavaScript. TypeScript обеспечивает проверку типов на этапе компиляции, что позволяет выявлять целый класс ошибок до запуска приложения, улучшает автодополнение в среде разработки и делает код более самодокументируемым. Для сборки проекта используется инструмент Vite, обеспечивающий мгновенный запуск в режиме разработки за счёт нативной поддержки ES-модулей и горячей замены модулей (HMR) без полной пересборки.

Для построения графиков аналитических данных рассматривались Recharts и Chart.js.

Recharts – это библиотека графиков для React, построенная поверх D3.js и использующая React-компоненты в качестве строительных блоков. Recharts предоставляет LineChart, BarChart, PieChart, RadarChart и другие типы диаграмм в декларативном стиле, совместимом с React-моделью рендеринга.

Chart.js – это библиотека для построения графиков, работающая через Canvas API. Chart.js не является React-нативной библиотекой и требует

использования обёрток (react-chartjs-2) для интеграции с React, что добавляет уровень косвенности.

Для разработки выбрана библиотека Recharts версии 3.8.1 как нативно интегрированная с React и обеспечивающая декларативный способ описания диаграмм.

Для формата Excel выбрана библиотека openpyxl – чистая Python-реализация без внешних зависимостей, поддерживающая создание многолистовых книг, применение стилей ячеек, автоматическую подстройку ширины столбцов и запись данных в потоковый буфер BytesIO. Для формата PDF выбрана библиотека ReportLab – библиотека программного создания PDF-документов на Python. ReportLab поддерживает создание таблиц с повторяющимися заголовками на каждой странице, настройку стилей абзацев и ячеек, выбор размера и ориентации страницы, а также потоковую генерацию в буфер BytesIO.

Для управления схемой базы данных и её миграциями используется Alembic – стандартный инструмент для SQLAlchemy-проектов, позволяющий описывать изменения схемы в виде последовательности версионированных скриптов и применять их к базе данных инкрементально.

4.2 ВЫБОР И ОБОСНОВАНИЕ ВЫБОРА ТЕХНИЧЕСКОГО ОБЕСПЕЧЕНИЯ

Разрабатываемая система является веб-приложением и не предъявляет особых требований к техническому обеспечению пользователя. Однако для стабильной работы клиентской части, серверной части и базы данных необходимо соответствие определённым минимальным характеристикам.

Требования к устройству пользователя-клиента (обучающегося, преподавателя или администратора со стороны клиентского интерфейса):

- операционная система: Windows 10 и выше, macOS 12 и выше, либо любой актуальный дистрибутив Linux для настольных устройств; для мобильных устройств – Android 10 и выше или iOS 15 и выше;
- объём оперативной памяти: не менее 2 ГБ;
- установленный браузер с поддержкой JavaScript ES2020 и современных Web API: Google Chrome версии 90 и выше, Mozilla Firefox

версии 90 и выше, Microsoft Edge версии 90 и выше или Safari версии 15 и выше;

- подключение к сети Интернет или локальной сети со скоростью не менее 5 Мбит/с;

- не менее 200 МБ свободного места на диске для хранения кэша браузера.

Никакого дополнительного программного обеспечения, плагинов или клиентских компонентов на устройстве пользователя не требуется: система полностью функционирует в браузере.

Требования к серверу приложений (Uvicorn + FastAPI):

- операционная система: Linux (рекомендуется Ubuntu 22.04 LTS или Debian 12) либо Windows Server 2019 и выше;

- процессор: не менее 2 ядер с тактовой частотой от 2,0 ГГц;

- объём оперативной памяти: не менее 4 ГБ (рекомендуется 8 ГБ при одновременной работе более 50 пользователей);

- объём дискового пространства: не менее 10 ГБ для хранения кода приложения, медиафайлов (изображений вопросов) и файлов виртуального окружения Python;

- установленный интерпретатор Python версии 3.12 и выше;

- сетевой доступ к серверу базы данных по порту 5432.

Требования к серверу базы данных (PostgreSQL):

- операционная система: Linux (рекомендуется Ubuntu 22.04 LTS) либо Windows Server 2019 и выше;

- процессор: не менее 2 ядер с тактовой частотой от 2,0 ГГц;

- объём оперативной памяти: не менее 4 ГБ (рекомендуется 8 ГБ при значительном объёме данных и интенсивных аналитических запросах);

- объём дискового пространства: не менее 20 ГБ для хранения данных базы, журналов транзакций и резервных копий;

- установленная СУБД PostgreSQL версии 15.0 и выше;

- установленные расширения: uuid-ossr (генерация UUID) и pg_trgm (нечёткий поиск по строкам).

В простейшей конфигурации сервер приложений и сервер базы данных могут быть развёрнуты на одном физическом или виртуальном сервере при условии соответствия суммарным требованиям к ресурсам.

ЗАКЛЮЧЕНИЕ

Целью работы являлась разработка первых двух разделов выпускной квалификационной работы по теме «Автоматизированная система мониторинга успеваемости обучающихся».

Для достижения поставленной цели в ходе преддипломной практики был выполнен следующий комплекс работ.

В рамках первого раздела проведен анализ предметной области: изучены особенности организации контроля знаний в образовательных организациях, рассмотрены концептуальные подходы к построению систем мониторинга успеваемости. Выполнен сравнительный анализ существующих программных решений – Яндекс Форм, системы MOODLE и сервиса Online Test Pad. Анализ позволил установить, что представленные на рынке системы не обеспечивают комплексного мониторинга динамики освоения учебного материала и не поддерживают адаптивный подбор заданий на основе статистики выполнения, что обосновывает необходимость собственной разработки.

В рамках второго раздела осуществлена постановка задачи на проектирование и разработку системы: определены основные характеристики разрабатываемой системы, ее объект и предмет, сформулированы ограничения и рекомендации, разработаны функциональные и нефункциональные требования. Определена структура входной и выходной информации: описаны данные об обучающихся, преподавателях, дисциплинах и результатах учебной деятельности, а также состав формируемых системой аналитических отчетов.

Таким образом, в ходе преддипломной практики были выполнены задания: подготовлены версии первого и второго разделов выпускной квалификационной работы, заложена основа для дальнейшей программной реализации, тестирования и внедрения автоматизированной системы мониторинга успеваемости.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Академическая успеваемость под контролем // Кузбасский государственный университет имени Т.Ф. Горбачева URL: <https://kuzstu.ru/news/8378/akademicheskaya-uspevaemost-pod-kontrolem/> (дата обращения: 18.02.2026);
2. Фирсова С.А., Рябухина Е.А., Гладунов Н.Д. Разработка программного модуля для управления образовательной деятельностью учреждений общего образования с использованием методов ABC- и XYZ-анализа успеваемости и посещаемости обучающихся. Информатика и образование. 2025;40(1):42-56. <https://doi.org/10.32517/0234-0453-2025-40-1-42-56> (дата обращения: 18.02.2026);
3. Чурилов Михаил Романович, Дугина Самира Юсифовна ВИДЫ И ФОРМЫ ПРОВЕРКИ ЗНАНИЙ, УМЕНИЙ И НАВЫКОВ ОБУЧАЮЩИХСЯ // Вестник НИБ. 2024. №54. URL: <https://cyberleninka.ru/article/n/vidy-i-formy-proverki-znaniy-umeniy-i-navykov-obuchayuschih-sya> (дата обращения: 21.02.2026);
4. Ахметова, Клара ОБЩАЯ ХАРАКТЕРИСТИКА МОНИТОРИНГА В ОБРАЗОВАНИИ // ORIENSS. 2023. №1-2. URL: <https://cyberleninka.ru/article/n/obschaya-harakteristika-monitoringa-v-obrazovanii> (дата обращения: 21.02.2026);
5. ЦИФРОВИЗАЦИЯ ОБРАЗОВАНИЯ КАК ГЛОБАЛЬНЫЙ ТРЕНД / Мусаева Амина Караматовна. – ИКРО журнал. – 2025. – № 02. – С. 1088-9. URL: <https://inlibrary.uz/index.php/igro/article/view/104768> (дата обращения: 21.02.2026);
6. Цифровизация образования // Минобрнауки России URL: https://www.minobrnauki.gov.ru/colleges_councils/kollegialnye-organy/digitalcouncil/digitalobr/ (дата обращения: 21.02.2026);
7. Чевычелов Ю. А., Шевырева Е. В. Использование методов мониторинга для улучшения успеваемости // Новая парадигма научного знания в цифровую эпоху : сборник научных трудов по материалам Международной научно-практической конференции 15 мая 2023г. Белгород : ООО Агентство перспективных научных исследований (АПНИ), 2023. С. 100-103. URL: <https://apni.ru/article/6209-ispolzovanie-metodov-monitoringa-dlya-uluchsh> (дата обращения: 21.02.2026);

8. Sáiz-Manzanares, M. C., Rodríguez-Díez, J. J., Díez-Pastor, J. F., Rodríguez-Arribas, S., Marticorena-Sánchez, R., & Ji, Y. P. (2021). Monitoring of Student Learning in Learning Management Systems: An Application of Educational Data Mining Techniques. *Applied Sciences*, 11(6), 2677. URL: <https://doi.org/10.3390/app11062677> (дата обращения: 21.02.2026);

9. Ерохин В.В. Автоматизация проведения мониторинга успеваемости студентов в вузах / В.В.Ерохин // Вестник современных исследований. – 2019. – №2-13 (29). – С. 7-14. URL: <https://www.elibrary.ru/item.asp?id=37126813> (дата обращения: 21.02.2026);